

Modélisation UML

Christine Solnon

INSA de Lyon - 3IF

2013 - 2014

Positionnement de l'UE / IF

Domaines d'enseignement du département IF :

- Système d'Information
- Réseaux
- Architectures matérielles
- Logiciel Système
- Méthodes et Outils Mathématiques
- Formation générale
- **Développement logiciel**

Unités d'Enseignement du domaine "Développement logiciel" :

- C++ (3IF)
- Génie logiciel (3IF)
- **Modélisation UML** (3IF)
- Qualité logiciel (4IF)
- Grammaires et langages (4IF)
- Ingénierie des IHM (4IF)
- Méthodologie de développement objet (4IF)

Référentiel des compétences

Utiliser des diagrammes UML pour modéliser un objet d'étude

- Interpréter un diagramme UML donné
→ **IF3-UML**, IF4-DevOO, IF4-IHM
- Concevoir un diagramme UML modélisant un objet d'étude
→ **IF3-UML**, IF3-C++, IF3-DASI, IF4-DevOO, IF4-IHM, IF4-LG
- Vérifier la cohérence de différents diagrammes modélisant un même objet d'étude
→ **IF3-UML**, IF4-DevOO, IF4-LG

Concevoir l'architecture d'un logiciel orienté objet

- Structurer un logiciel en paquetages et classes faiblement couplés et fortement cohésifs
→ **IF3-UML**, IF3-C++, IF3-DASI, IF4-DevOO, IF4-LG
- Utiliser des Design Patterns
→ **IF3-UML**, IF3-C++, IF3-DASI, IF4-DevOO, IF4-LG

Organisation

6 séances de cours

- du 7 au 28 novembre

4 séances de travaux dirigés (TD)

- du 18 novembre au 18 décembre

1 devoir surveillé (DS)

- le 23 janvier

Pour en savoir plus...

- **Sur la modélisation en général :**

- *Modèles et Métamodèles*
Guy Caplat

- **Sur le méta-modèle de référence d'UML :**

- <http://www.omg.org/uml>

- **Sur UML et la modélisation objet :**

- *Modélisation Objet avec UML*
Pierre-Alain Muller, Nathalie Gaertner

→ **Chapitre sur la notation téléchargeable sur le site d'Eyrolles**

- **Sur les design patterns et la conception orientée objet :**

- *UML 2 et les design patterns*
Craig Larman
- *Tête la première : Design Patterns*
Eric Freeman & Elizabeth Freeman

- **...et plein d'autres ouvrages à Doc'INSA !**

Plan du cours

1

Introduction

- Introduction à la modélisation
- Introduction à UML

2

Modéliser la structure avec UML

3

Modéliser le comportement avec UML

4

Principes et patrons de conception orientée objet

Qu'est-ce qu'un modèle ?

Modèle = Objet conçu et construit (artefact) :

- Pour représenter un sujet d'études

Représentativité

Exemple de sujet : les circuits électriques

- S'appliquant à plusieurs cas de ce sujet d'étude

Généricité

Exemple de cas : des mesures (tension, intensité, ...) sur des circuits

- Incarnant un point de vue sur ces cas

Abstraction

Exemple de point de vue : $U = RI$

↪ Abstraction de la longueur des fils, la forme du circuit, ...

Un même sujet d'études peut avoir plusieurs modèles

↪ Chaque modèle donne un point de vue différent sur le sujet

Langages de modélisation

Langages utilisés pour exprimer un modèle :

- Langues naturelles : qui évoluent hors du contrôle d'une théorie
Ex : Français, Anglais, ...
- Langues artificiels : conçus pour des usages particuliers
 - Langues formels : syntaxe définie par une grammaire
Ex : Logique, langages informatique (C, Java, SQL, ...), ...

Pouvoir d'expression d'un langage :

↪ Ensemble des modèles que l'on peut exprimer

- Le choix du langage influence la conception du modèle...
...et donc la perception du sujet d'études !

Interprétation d'un langage :

↪ Procédure pour comprendre un modèle (Sémantique)

- Modèle ambigu : Plusieurs interprétations différentes possibles
- Modèle exécutable : Interprétation exécutable par une machine

Langages de modélisation basés sur les graphes

Définition

Un graphe est défini par un couple (N, A) tel que

- N est un ensemble de nœuds (aussi appelés sommets)
 $\leadsto$ Composants du modèle
- $A \subseteq N \times N$ est un ensemble d'arcs
 $\leadsto$ Relation binaire entre les composants du modèle

Nœuds et arcs peuvent être étiquetés par des propriétés

La modélisation par les graphes date de [Euler 1735]



[Image empruntée à Wikipedia]

Plan du cours

1 Introduction

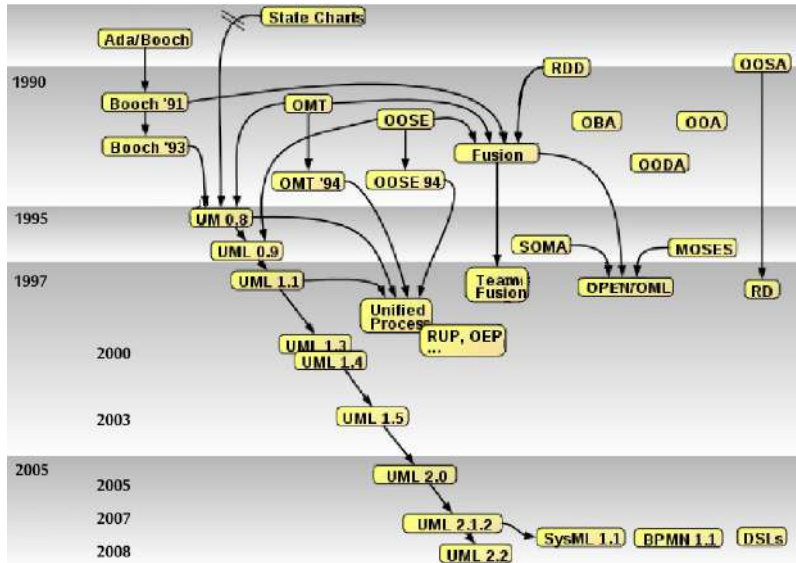
- Introduction à la modélisation
- Introduction à UML

2 Modéliser la structure avec UML

3 Modéliser le comportement avec UML

4 Principes et patrons de conception orientée objet

Historique d'UML



[Image empruntée à Wikipedia]

UML et l'OMG

OMG = Object Management Group (www.omg.org) :

- Fondé en 1989 pour standardiser et promouvoir l'objet
- Version 1.0 d'UML (Unified Modeling Language) en janvier 1997
- Version 2.5 en octobre 2012

Définition d'UML selon l'OMG :

Langage visuel dédié à la spécification, la construction et la documentation des artefacts d'un système logiciel

L'OMG définit le méta-modèle d'UML

→ Syntaxe et interprétation en partie formalisées

Attention : UML est un langage... pas une méthode

→ Méthode dans le cours 4IF "Développement Orienté Objet"

3 façons d'utiliser UML selon [Fowler 2003]

(On y reviendra en 4IF...)

Mode esquisse (méthodes Agile) :

- Diagrammes tracés à la main, informels et incomplets

~> Support de communication pour concevoir les parties critiques

Mode plan :

- Diagrammes formels relativement détaillés
- Annotations en langue naturelle

~> Génération d'un squelette de code à partir des diagrammes

~> Nécessité de compléter le code pour obtenir un exécutable

Mode programmation (Model Driven Architecture / MDA) :

- Spécification complète et formelle en UML

~> Génération automatique d'un exécutable à partir des diagrammes

~> Limité à des applications bien particulières

~> Un peu utopique (...pour le moment ?)

Différents modèles UML

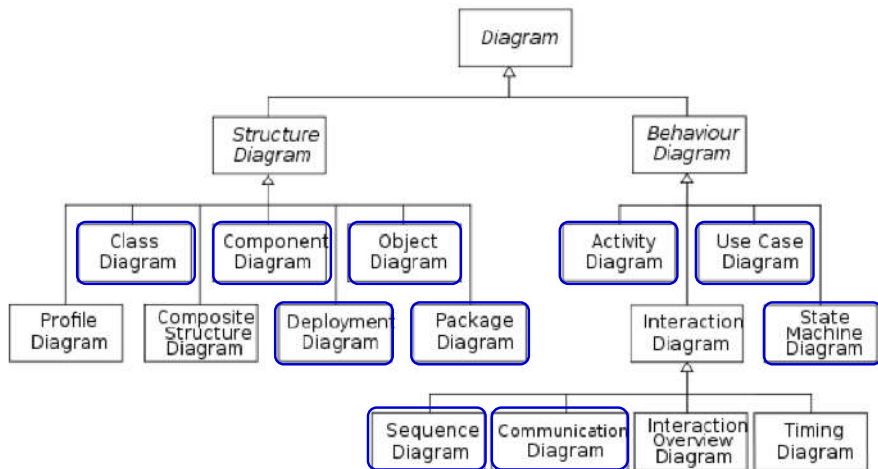
UML peut être utilisé pour définir de nombreux modèles :

- Modèles descriptifs vs prescriptifs
 - Descriptifs $\leadsto$ Décrire l'existant (domaine, métier)
 - Prescriptifs $\leadsto$ Décrire le futur système à réaliser
- Modèles destinés à différents acteurs
 - Pour l'utilisateur $\leadsto$ Décrire le quoi
 - Pour les concepteurs/développeurs $\leadsto$ Décrire le comment
- Modèles statiques vs dynamiques
 - Statiques $\leadsto$ Décrire les aspects structurels
 - Dynamiques $\leadsto$ Décrire comportements et interactions

Les modèles sont décrits par des diagrammes (des graphes)

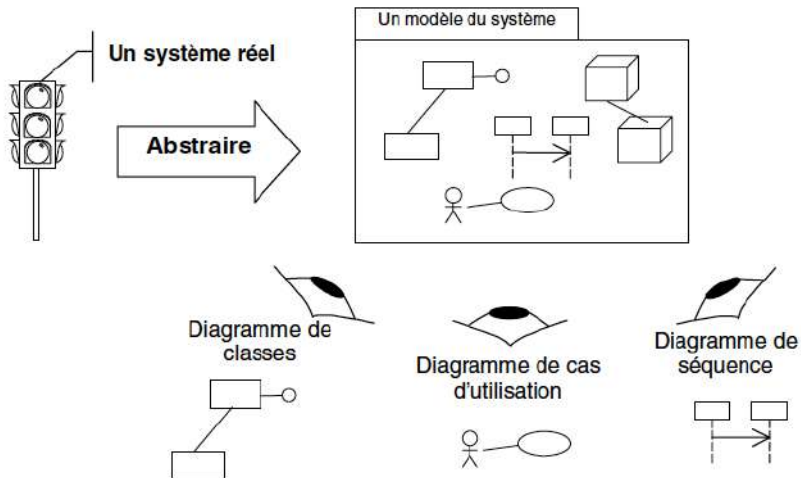
$\leadsto$ Chaque diagramme donne un point de vue différent sur le système

14 types de diagrammes d'UML 2.2



...et un langage pour exprimer des contraintes : OCL

Diagrammes UML : Points de vue sur le système



[Image empruntée à Muller et Gaertner]

Notations communes à tous les diagrammes (1/2)

Stéréotypes et mots-clés (on y reviendra en 4IF...) :

- Définition d'une utilisation particulière d'éléments de modélisation
 ~> Interprétation (sémantique) particulière
- Notation : «nomDuStéréotype» ou {nomDuMotClé} ou icône
- Nombreux stéréotypes et mots-clés prédéfinis : «interface», «invariant», «create», «actor», {abstract}, {bind}, {use}...

Valeurs marquées (on y reviendra aussi en 4IF...) :

- Ajout d'une propriété à un élément de modélisation
- Notation : { nom₁ = valeur₁, ..., nom_n = valeur_n}
- Valeurs marquées prédéfinies (ex. : derived : Bool)
 ou personnalisées (ex. : auteur : Chaîne)

Commentaires :

- Information en langue naturelle
- Notation : o-----



Notations communes à tous les diagrammes (2/2)

Relations de dépendance :

- Notation : `[source]----->[cible]`
~> Modification de la source peut impliquer une modification de la cible
- Nombreux stéréotypes prédéfinis : `«bind»`, `«realize»`, `«use»`, `«create»`, `«call»`, ...

Contraintes (on y reviendra aussi en 4IF) :

- Relations entre éléments de modélisation
~> Propriétés qui doivent être vérifiées
~> Attention : les contraintes n'ont pas d'effet de bord
- Notation : `o-----{ contrainte }`
3 types de contraintes :
 - Contraintes prédéfinies : `disjoint`, `ordered`, `xor`, ...
 - Contraintes exprimées en langue naturelle
Ex. : `temps d'exécution inférieur à 10 ms`
 - Contraintes exprimées avec OCL (Object Constraint Language)
Ex. : `context Pile inv : self.nbElts >= 0`
- Stéréotypes : `«invariant»`, `«précondition»`, `«postcondition»`

Plan du cours

- 1 Introduction
- 2 Modéliser la structure avec UML**
- 3 Modéliser le comportement avec UML
- 4 Principes et patrons de conception orientée objet

Point de vue statique sur le système

Décrire la structure du système en termes de :

- Composants du système
 ~> Objets, Classes, Paquetages, Composants, ...
- Relations entre ces composants
 ~> Spécialisation, Association, Dépendance, ...

~> Pas de facteur temps

Différents diagrammes statiques que nous allons voir :

- Diagrammes d'objets (Cours)
- Diagrammes de classes (Cours + TD)
- Diagrammes de paquetage (Cours + TD)
- Diagrammes de composants (Cours)
- Diagrammes de déploiement (Cours)

Plan du cours

- 1 Introduction
- 2 **Modéliser la structure avec UML**
 - Structuration Orientée Objet
 - Diagrammes d'objets
 - Diagrammes de classes
 - Diagrammes de paquetage
 - Diagrammes de composants
 - Diagrammes de déploiement
- 3 **Modéliser le comportement avec UML**
- 4 **Principes et patrons de conception orientée objet**

Pourquoi une structuration orientée objet ?

Unicité et universalité du paradigme

- Réduire le décalage entre monde réel et logiciel

↪ Objets réels $\Rightarrow$ Objets conceptuels $\Rightarrow$ Objets logiciels

Réutilisabilité et évolutivité facilitées par différents mécanismes

- Encapsulation, Modularité, Abstraction, Polymorphisme, Héritage

↪ Faible couplage inter-objets / Forte cohésion intra-objet

Paradigme qui arrive à maturité

- Bibliothèques de classes, Design patterns, UML, Méthodologies de développement (UP, Agile, XP, ...), ...

↪ Environnements de développement intégrés (IDE)

Qu'est-ce qu'un objet ?

Objet = Etat + Comportement + Identité

Etat d'un objet :

- Ensemble de valeurs décrivant l'objet
 - ↪ Chaque valeur est associée à un attribut (propriété)
 - ↪ Les valeurs sont également des objets (⇒ liens entre objets)

Comportement d'un objet :

- Ensemble d'opérations que l'objet peut effectuer
- Chaque opération est déclenchée par l'envoi d'un message
 - ↪ Exécution d'une méthode

Identité d'un objet :

- Permet de distinguer les objets indépendamment de leur état
 - ↪ 2 objets différents peuvent avoir le même état
- Attribuée implicitement à la création de l'objet
 - ↪ L'identité d'un objet ne peut être modifiée

Attention : identité $\neq$ nom de la variable qui référence l'objet

Qu'est ce qu'une classe d'objets ?

Classe = regroupement d'objets similaires (appelés instances)

- Toutes les instances d'une classe ont les mêmes attributs et opérations
~> Abstraction des caractéristiques non communes

Classes sans instance

- Classes abstraites :
~> Certaines opérations peuvent être abstraites/virtuelles (non définies)
- Interfaces :
~> Pas d'attribut et toutes les opérations sont abstraites/virtuelles

Relation de spécialisation/généralisation entre classes

- Une classe A est une spécialisation d'une classe B si tout attribut/opération de B est également attribut/opération de A
- Implémentation par héritage

Plan du cours

- 1 Introduction
- 2 **Modéliser la structure avec UML**
 - Structuration Orientée Objet
 - Diagrammes d'objets
 - Diagrammes de classes
 - Diagrammes de paquetage
 - Diagrammes de composants
 - Diagrammes de déploiement
- 3 **Modéliser le comportement avec UML**
- 4 **Principes et patrons de conception orientée objet**

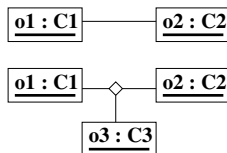
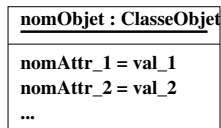
Diagrammes d'objets

Objectif : Représenter les objets et leurs liens à un instant donné

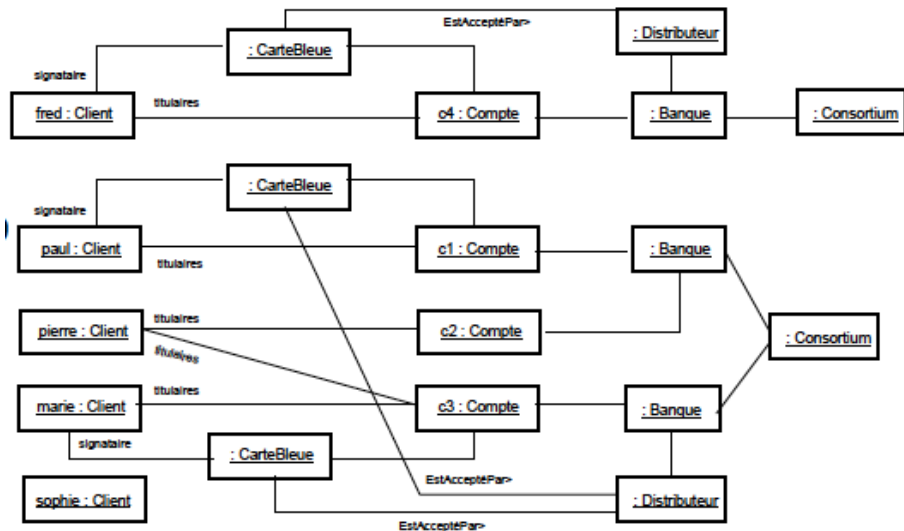
- Utilisation : documenter des cas de test, analyser des exemples, ...

Moyen : Graphe

- Nœuds du graphe = Objets
 - Possibilité de supprimer le nom, la classe et/ou les attributs (objet anonyme, non typé ou d'état inconnu)
- Arêtes du graphe = Liens entre objets
 - Lien binaire : entre 2 objets
 - Lien n-aire : entre n objets
 - Possibilité de nommer les liens et les rôles
- Correspondance entre liens et attributs



Exemple de diagramme d'objets



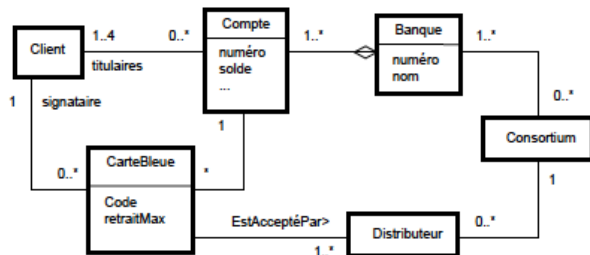
Au travail !

- Pierre, Paul, Jacques, Marie et Anne sont étudiants au département IF.
- Robert et Suzie sont enseignants au département IF.
- Robert enseigne le C et le réseau ;
Suzie enseigne l'anglais, les math et le Java.
- Pierre, Paul et Marie suivent les cours de C, de réseau et Java ;
Jacques et Anne suivent les cours de C, math et anglais.

Plan du cours

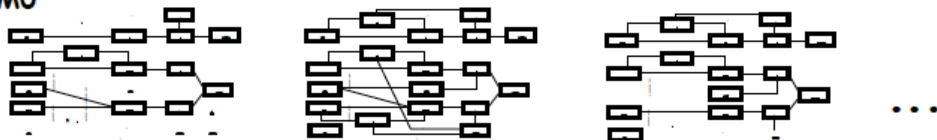
- 1 Introduction
- 2 **Modéliser la structure avec UML**
 - Structuration Orientée Objet
 - Diagrammes d'objets
 - Diagrammes de classes
 - Diagrammes de paquetage
 - Diagrammes de composants
 - Diagrammes de déploiement
- 3 **Modéliser le comportement avec UML**
- 4 **Principes et patrons de conception orientée objet**

Abstraction d'un ensemble de diagrammes d'objets



M1

M0



[Image empruntée à J.-M. Jezequel]

Diagrammes de classes

Un diagramme de classes est un graphe :

- Nœud du graphe = Classe
 - ↪ Abstraction d'un ensemble d'objets
- Arc du graphe = Relation entre des classes :
 - Relation d'association
 - ↪ Abstraction d'un d'ensemble de liens entre objets
 - Relation de généralisation / spécialisation
 - ↪ Factorisation de propriétés communes à plusieurs classes

Très nombreuses utilisations, à différents niveaux :

- Pendant la capture des besoins : Modèle du domaine
 - ↪ Classes = Objets du domaine
- Pendant la conception/implémentation : Modèle de conception
 - ↪ Classes = Objets logiciels

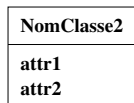
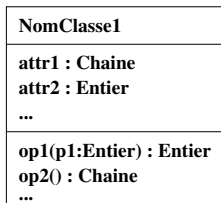
Représentation UML des classes

Rectangle composé de compartiments :

- Compartiment 1 : Nom de la classe (commence par une majuscule, en gras)
- Compartiment 2 : Attributs
- Compartiment 3 : Opérations
- Possibilité d'ajouter des compartiments (exceptions, ...)

Différents niveaux de détail possibles :

~> Possibilité d'omettre attributs et/ou opérations



Représentation UML des attributs

- Format de description d'un attribut :

[Vis] Nom [Mult] [":" TypeAtt] ["=" Val] [Prop]

- Vis : + (public), - (privé), # (protégé), ~ (package)
 - Mult : "[" nbElt "]" ou "[" Min .. Max "]"
 - TypeAtt : type primitif (Entier, Chaîne, ...) ou classe
 - Val : valeur initiale à la création de l'objet
 - Prop : {gelé}, {variable}, {ajoutUniquement}, ...
- Attributs de classe (statiques) soulignés
 - Attributs dérivés précédés de "/"

Exemples :

- | | |
|--------------------|-----------------------------|
| ● # onOff : Bouton | ● + pi : réel = 3.14 {gelé} |
| ● - x : Réel | ● inscrits[2..8] : Personne |
| ● coord[3] : Réel | ● /age : Entier |

Représentation UML des opérations

- Format de description d'une opération :

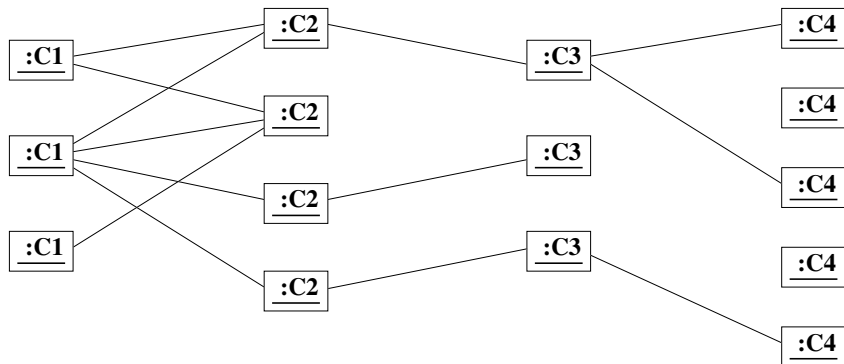
[Visibilité] Nom ["(" Arg ")"] [":" Type]

- Visibilité : + (public), - (privé), # (protégé)
- Arg : liste des arguments selon le format
[Dir] NomArgument : TypeArgument
où Dir = in (par défaut), out, ou inout
- Type : type de la valeur retournée (type primitif ou classe)
- Opérations abstraites/virtuelles (non implémentées) en italique
- Opérations de classe (statiques) soulignées
- Possibilité d'annoter avec des contraintes stéréotypées
«precondition» et «postcondition»
~> Programmation par contrats
- Possibilité de surcharger une opération :
~> même nom, mais paramètres différents
- Stéréotypes d'opérations : «create» et «destroy»

Associations entre classes

~ Abstraction des relations définies par les liens entre objets

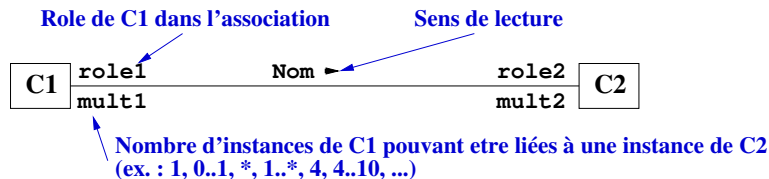
Liens entre objets :



Associations entre classes d'objets :



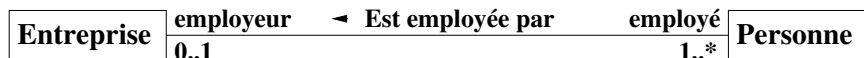
Associations entre classes (1/2)



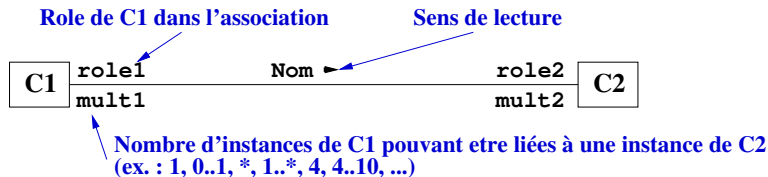
Interprétation en français :

- Un C1 `Nom` un C2 (ou un C2 `Nom` un C1 si sens de lecture inverse)
- Un C1 est `rôle1` d'un C2 et `mult1` C1 peuvent jouer ce rôle pour un C2
- Un C2 est `rôle2` d'un C1 et `mult2` C2 peuvent jouer ce rôle pour un C1

Exemple :



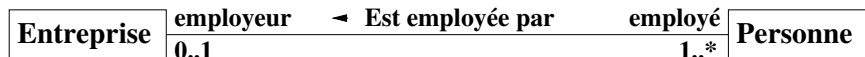
Associations entre classes (2/2)



Interprétation en langage de programmation orienté objet

- La classe C1 a un attribut de nom `rôle2`
 $\leadsto$ Type = C2 si `mult2` $\in$ {1, 0..1}, ou collection de C2 sinon
- La classe C2 a un attribut de nom `rôle1`
 $\leadsto$ Type = C1 si `mult1` $\in$ {1, 0..1}, ou collection de C1 sinon

Exemple :



Au travail !

- Chaque étudiant du département IF suit un ensemble d'unités d'enseignement (UE).
- Chaque UE a un coefficient et est constituée de cours, de travaux dirigés (TD) et de travaux pratiques (TP).
- Chaque cours, TD ou TP a une date. Les cours sont faits en amphi, les TD en salle de classe et les TP en salle machine.
- Pour les TP et TD, les étudiants sont répartis dans des groupes. Pour chaque TP, chaque étudiant est en binôme avec un autre étudiant.
- Les cours et les TD sont assurés par un enseignant. Les TP sont assurés par deux enseignants.
- Pour chaque UE, l'étudiant a une note de devoir surveillé ; pour chaque TP, le binôme a une note de TP.

Navigabilité

Qu'est-ce que la navigabilité d'une association entre C1 et C2 ?

Capacité d'une instance de C1 (resp. C2) à accéder aux instances de C2 (resp. C1)

Par défaut :

Navigabilité dans les deux sens

→ C1 a un attribut de type C2 et C2 a un attribut de type C1



Spécification de la navigabilité :

Orientation de l'association

→ C1 a un attribut du type de C2, mais pas l'inverse



Attention :

Dans un diagramme de classes conceptuelles, toute classe doit être accessible à partir de la classe principale

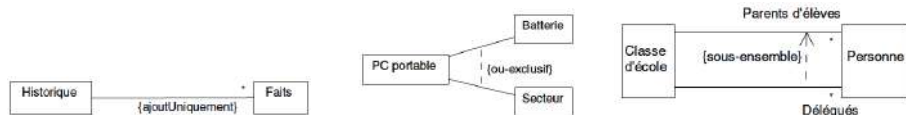
Propriétés et contraintes sur les associations

Propriétés sur extrémités d'associations

- `{variable}` : instance modifiable (par défaut)
- `{frozen}` : instance non modifiable
- `{addOnly}` : instances ajoutables mais non retirables (si mult. > 1)

Contraintes prédéfinies

- Sur une extrémité : `{ordered}`, `{unique}`, ...
- Entre 2 associations : `{subset}`, `{xor}`, ...

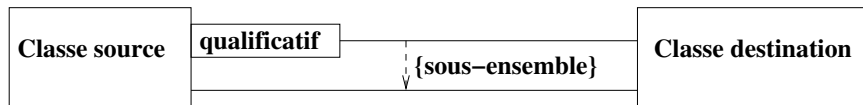


[images extraites de Muller et Gaertner]

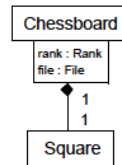
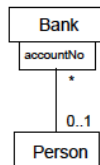
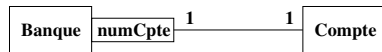
Associations qualifiées

Restriction d'une relation / qualificateur :

- Sélection d'un sous-ensemble d'objets à l'aide d'un attribut qualificatif (clé)
- L'attribut qualificatif appartient à l'association



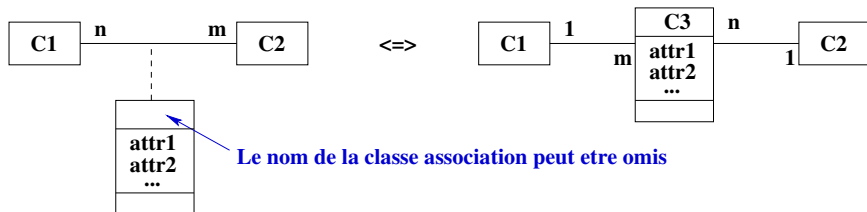
Exemples :



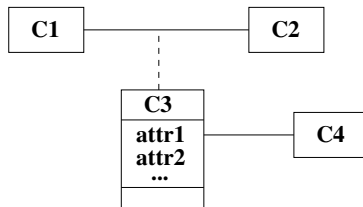
[image extraite de www.omg.org]

Classes-associations

Association attribuée :

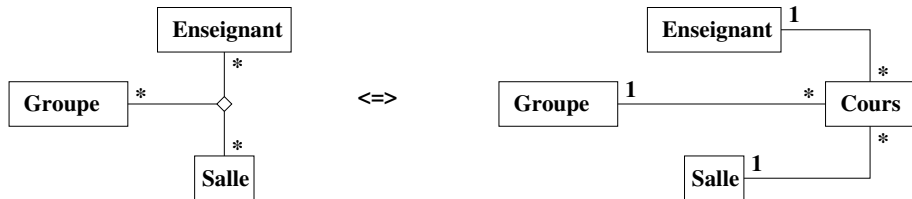


Une classe association peut participer à d'autres associations :

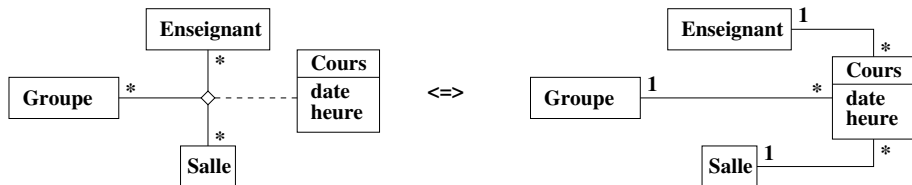


Associations n-aires

Associations entre n classes (avec $n > 2$)

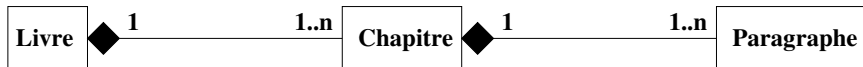


Classes-Associations n-aire



Associations particulières : composition et agrégation

Composition :



- Relation transitive et antisymétrique
- La création (copie, destruction) du composite (container) implique la création (copie, destruction) de ses composants
- Un composant appartient à au plus un composite

Agrégation :



- Simple regroupement de parties dans un tout

Généralisation et Héritage

Niveau conceptuel : Généralisation

- Relation transitive, non réflexive, et non symétrique
- La sous-classe “est-une-sort-de” la super classe
 ~> Toute instance de la sous-classe est instance de la super classe

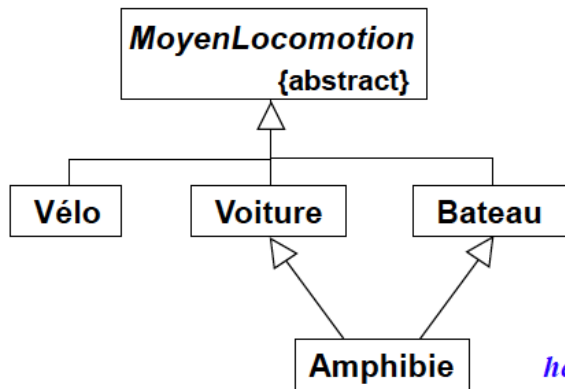
Niveau implémentation : Héritage

- Mécanisme proposé par les langages de programmation Objet
- "B hérite de A" signifie que B possède :
 - Toutes les propriétés de A (attributs, op., assoc., contraintes)
 ~> Possibilité de redéfinir les opérations de la sous-classe
 ~> Polymorphisme
 - Ainsi que des nouvelles propriétés qui lui sont propres

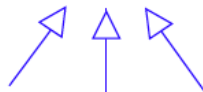
~> Permet de factoriser les propriétés communes à plusieurs classes

~> Une opération définie pour A est accessible aux sous-classes de A

Héritage et généralisation : Exemple



notation équivalente



héritage multiple

Héritage vs Composition

Problème :

Modéliser le fait qu'il y a des voitures bleues, des voitures rouges et des voitures vertes.

Solution 1 : Héritage

- Créer une classe abstraite Voiture
- Créer 3 classes VoitureBleue, VoitureRouge et VoitureVerte qui héritent de Voiture

Solution 2 : Composition

- Créer une classe Voiture et une classe Couleur (énumération)
- Créer une association entre Voiture et Couleur

Comment représenter ces 2 solutions en UML ?

Héritage vs Composition

Problème :

Modéliser le fait qu'il y a des voitures bleues, des voitures rouges et des voitures vertes.

Solution 1 : Héritage

- Créer une classe abstraite Voiture
- Créer 3 classes VoitureBleue, VoitureRouge et VoitureVerte qui héritent de Voiture

Solution 2 : Composition

- Créer une classe Voiture et une classe Couleur (énumération)
- Créer une association entre Voiture et Couleur

Comment représenter ces 2 solutions en UML ?

Quelle solution choisissez-vous ?

Héritage vs Composition

Problème :

Modéliser le fait qu'il y a des voitures bleues, des voitures rouges et des voitures vertes.

Solution 1 : Héritage

- Créer une classe abstraite Voiture
- Créer 3 classes VoitureBleue, VoitureRouge et VoitureVerte qui héritent de Voiture

Solution 2 : Composition

- Créer une classe Voiture et une classe Couleur (énumération)
- Créer une association entre Voiture et Couleur

Comment représenter ces 2 solutions en UML ?

Quelle solution choisissez-vous ?

Et si on veut modéliser le fait qu'il y a des personnes hommes et des personnes femmes ?

Héritage vs Délégation

Problème :

Appeler dans une classe B une opération `op()` d'une classe A ?

Solution 1 : Héritage

- Faire hériter B de A
- `op()` peut être appelée depuis n'importe quelle instance de B

Solution 2 : Délégation

- Ajouter une association de B vers A
 $\leadsto$ Ajouter dans B un attribut `a` de type A
- `a.op()` peut être appelée depuis n'importe quelle instance de B

Comment représenter ces 2 solutions en UML ?

Héritage vs Délégation

Problème :

Appeler dans une classe B une opération `op()` d'une classe A ?

Solution 1 : Héritage

- Faire hériter B de A
- `op()` peut être appelée depuis n'importe quelle instance de B

Solution 2 : Délégation

- Ajouter une association de B vers A
 $\leadsto$ Ajouter dans B un attribut `a` de type A
- `a.op()` peut être appelée depuis n'importe quelle instance de B

Comment représenter ces 2 solutions en UML ?

Quelle solution choisissez-vous ?

Héritage vs Délégation

Problème :

Appeler dans une classe B une opération `op()` d'une classe A ?

Solution 1 : Héritage

- Faire hériter B de A
- `op()` peut être appelée depuis n'importe quelle instance de B

Solution 2 : Délégation

- Ajouter une association de B vers A
 $\leadsto$ Ajouter dans B un attribut `a` de type A
- `a.op()` peut être appelée depuis n'importe quelle instance de B

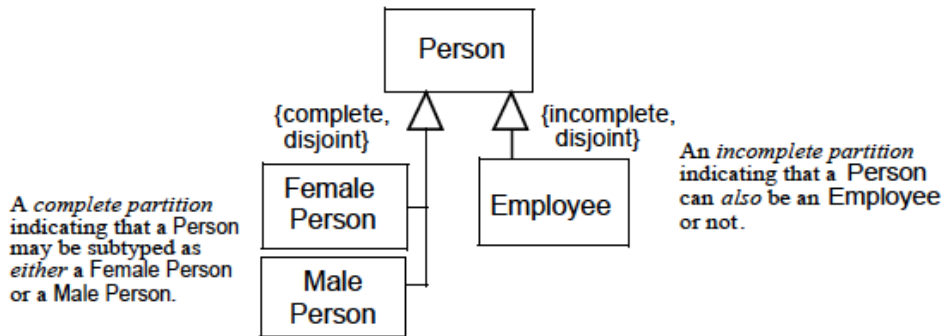
Comment représenter ces 2 solutions en UML ?

Quelle solution choisissez-vous ?

$\leadsto$ On y reviendra avec les Design Patterns...

Contraintes sur les relations de généralisation

- {disjoint} ou {overlapping}
- {complete} ou {incomplete}
 $\leadsto$ {disjoint, complete} $\Rightarrow$ Partition
- {leaf} ou {root}

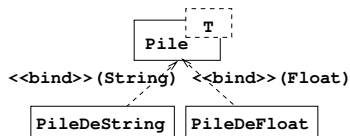


[Image empruntée à www.omg.org]

Classes génériques (templates)

Qu'est-ce qu'une classe générique ?

- Classe paramétrée par d'autres classes
 ~ Factorisation de code



Exemple de code C++

```

template <class T> class Pile{
public:
    Pile() { ... }
    void empile(T e) { ... }
    ...
private: ...
};
...
Pile<float> p1;
Pile<string> p2;
p1.empile(2.5);
p2.empile('a');
...
  
```

Exemple de code Java :

```

public class Pile<T> {
    public Pile() { ... }
    public void empile(T e) { ... }
    ...
    private ...
};
...
Pile<Float> p1 = new Pile<Float>();
Pile<String> p2 = new Pile<String>();
p1.empile(2.5);
p2.empile('a');
...
  
```

Classes abstraites

Qu'est-ce qu'une classe abstraite ?

- Classe qui ne peut être instanciée
 - Contient des opérations non définies :
 ↪ `abstract` (Java) ou `virtual` (C++)
 - Doit être spécialisée en une ou plusieurs classes non abstraites
- Notation : mot clé `{abstract}` ou nom en italique

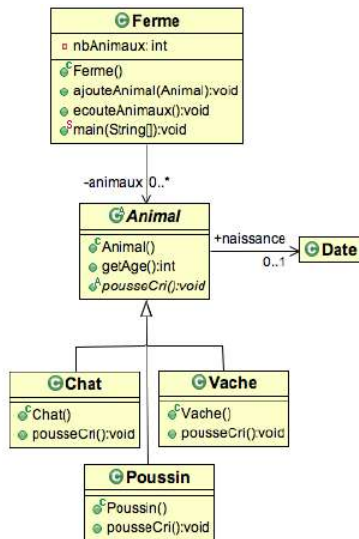
Pourquoi des classes abstraites ?

- Spécifier un comportement commun à plusieurs classes
- Manipuler des instances de classes différentes de façon uniforme
 ↪ Polymorphisme

Bonne pratique :

Dans une hiérarchie d'héritage, les classes qui ne sont pas des feuilles sont généralement abstraites

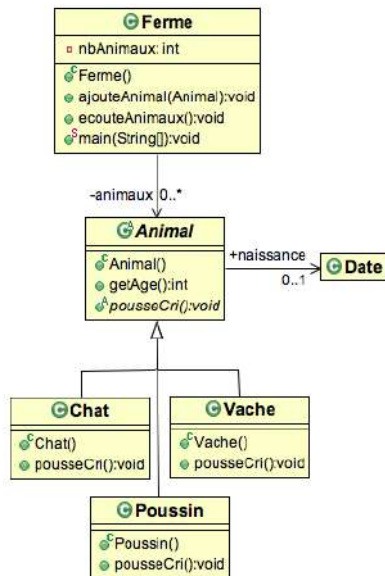
Classes abstraites et polymorphisme / Ex. C++



```

class Animal{
public:
    Date naissance;
    int getAge(){...}
    virtual void pousseCri() = 0;
};
class Poussin: public Animal{
public:
    void pousseCri(){cout << "Piou" << endl;}
};
class Ferme{
private:
    Animal* animaux[];
    int nbAnimaux;
public:
    Ferme(){...}
    void ajouteAnimal(Animal* a){
        animaux[nbAnimaux++] = a;
    }
    void ecouteAnimaux(){
        for (int i=0; i<nbAnimaux; i++)
            animaux[i]->pousseCri();
    }
};
int main(){
    Ferme f;
    f.ajouteAnimal(new Chat());
    f.ajouteAnimal(new Poussin());
    f.ajouteAnimal(new Vache());
    f.ecouteAnimaux();
}
  
```


Classes abstraites et polymorphisme / Ex. Java



```

public abstract class Animal {
    public Date naissance;
    public Animal(){...}
    public int getAge(){...}
    public abstract void pousseCri();
}

public class Poussin extends Animal {
    public void pousseCri() {System.out.println("Piou");}
}

public class Ferme {
    private Animal[] animaux;
    private int nbAnimaux;
    public Ferme(){...}
    public void ajouteAnimal(Animal a){
        animaux[nbAnimaux++] = a;
    }
    public void ecouteAnimaux(){
        for (int i=0; i<nbAnimaux; i++)
            animaux[i].pousseCri();
    }
}

public static void main(String[] args) {
    Ferme f = new Ferme();
    f.ajouteAnimal(new Chat());
    f.ajouteAnimal(new Poussin());
    f.ajouteAnimal(new Vache());
    f.ecouteAnimaux();
}
  
```

Interface

Qu'est-ce qu'une interface ?

- Classe sans attribut dont toutes les opérations sont abstraites
 - ~ Ne peut être instanciée
 - ~ Doit être **réalisée** (implémentée) par des classes non abstraites
 - ~ Peut hériter d'une autre interface

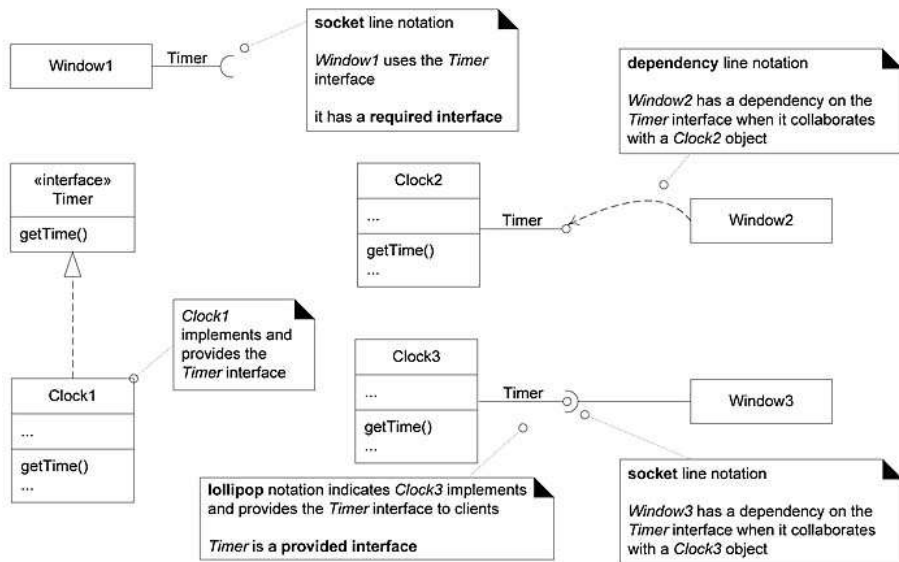
Pourquoi des interfaces ?

- Utilisation similaire aux classes abstraites
- En Java : une classe ne peut hériter de plus d'une classe, mais elle peut réaliser plusieurs interfaces

Notations UML :

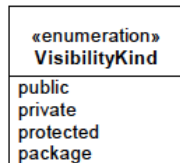
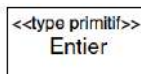
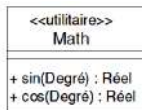
- Nom : «interface» Itf1
- Héritage : Itf1 $\rightarrow$ Itf2
- Réalisation : Class1 - - - $\rightarrow$ Itf1 ou Class1 $\xrightarrow{\text{Itf1}}$ O
- Utilisation : Class2 - - «use» - - $\rightarrow$ Itf1 ou Class2 $\xrightarrow{\text{Itf1}}$ C

Exemple



Quelques stéréotypes et mots-clés

- {abstract} : classe abstraite (alternative : nom en italique)
- «interface» : interface
- «énumération» : instances appartiennent à un ensemble fini de littéraux
- «type primitif» : instances sont des valeurs d'un type primitif
- «classe implémentation» : implémentation d'une classe dans un langage de programmation
- «utilitaire» : variables et procédures globales



Synthèse des différentes relations

- Association

- Navigable dans les 2 sens : 
- Navigable dans un sens : 
 - ou 

- Composition :

- Agrégation :

- Généralisation :

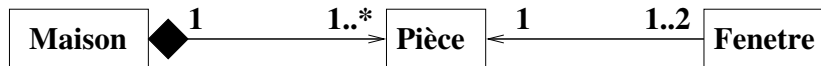
- Réalisation d'une interface : ou

- Utilisation d'une interface : ou

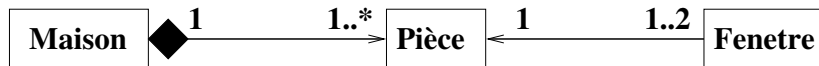
- Intanciation d'une classe générique :

- Dépendance :

Cherchez l'erreur (1/3)

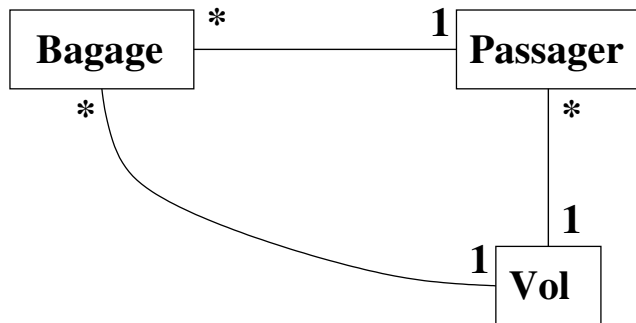


Cherchez l'erreur (1/3)

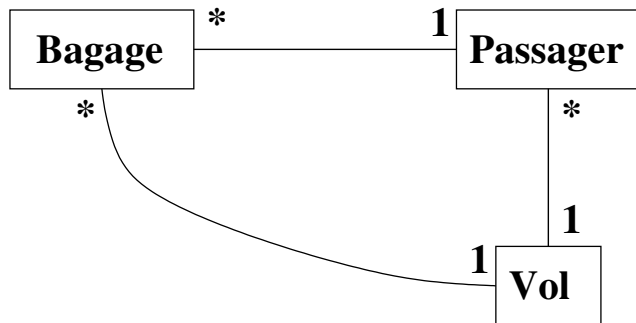


Navigabilité : Tout objet doit être accessible via une association

Cherchez l'erreur (2/3)

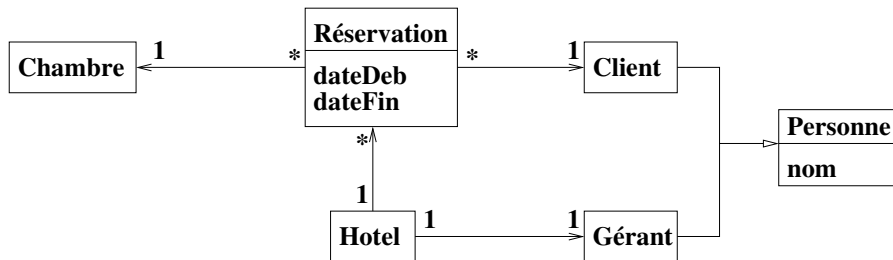


Cherchez l'erreur (2/3)

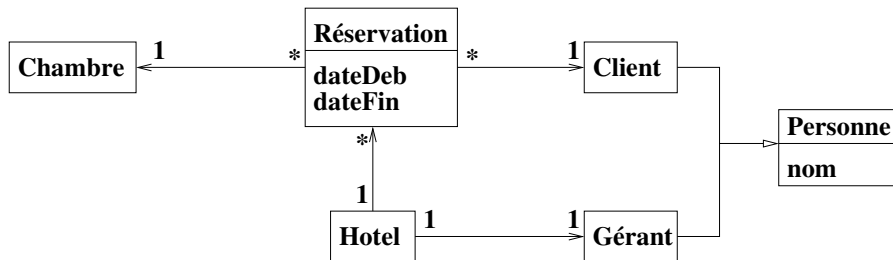


- Eviter les associations redondantes :
 - ↪ Le vol d'un bagage est obtenu à partir du passager
 - ↪ Les bagages d'un vol sont obtenus à partir des passagers
- Possibilité de spécifier que l'association Bagage-Vol est dérivée...

Cherchez l'erreur (3/3)



Cherchez l'erreur (3/3)



Navigabilité : En l'absence de réservations, on ne peut accéder ni aux chambres ni aux clients

Plan du cours

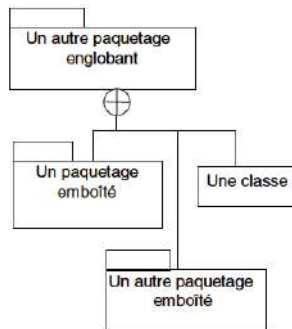
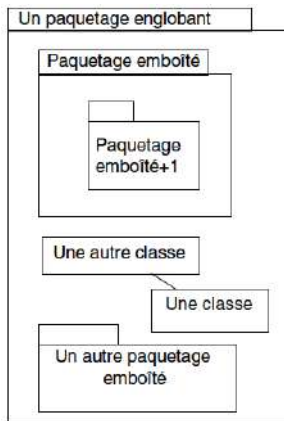
- 1 Introduction
- 2 **Modéliser la structure avec UML**
 - Structuration Orientée Objet
 - Diagrammes d'objets
 - Diagrammes de classes
 - **Diagrammes de paquetage**
 - Diagrammes de composants
 - Diagrammes de déploiement
- 3 **Modéliser le comportement avec UML**
- 4 **Principes et patrons de conception orientée objet**

Diagrammes de paquetages

Qu'est-ce qu'un paquetage (package) ?

Élément de modélisation qui :

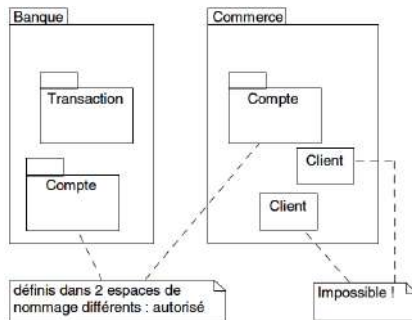
- Contient d'autres éléments de modélisation (classes, autres paquetages, ...)
→ Possibilité de ne pas représenter tous les éléments contenus
- Définit un espace de nom (namespace)



[Image extraite de Muller et Gaertner]

Espaces de nom et visibilité

- 2 éléments dans 2 paquetages différents sont différents, quel que soit leur nom



- Nom complet = nom préfixé par les noms des paquetages englobants : $\text{Banque}::\text{Compte} \neq \text{Commerce}::\text{Compte}$
- Visibilité d'un élément E dans un package P :
 - Public (+) : Élément visible par tous (utilisation du nom complet)
 - Privé (-) : Élément visible uniquement par
 - Les autres éléments de P
 - Les éléments englobés par E (si E est un package)

Dépendances entre paquetages

Reflètent les dépendances entre éléments des paquetages :

Une classe A dépend d'une classe B (noté A - - - - - > B) si :

- Il existe une association navigable de A vers B
(ou A possède un attribut de type B)
- Une méthode de A a un paramètre ou une variable locale de type B

Séréotypes sur les dépendances inter-paquetage

- A - - - «accède» - - - > B :
Tout élément public de B est accessible par son nom complet depuis A
- A - - - «importe» - - - > B :
Tout élément public de B est accessible par son nom depuis A
~> Création d'alias si nécessaire

Valeur marquée {global} :

Paquetage visible par tous les autres paquetages

~> Inutile de montrer les dépendances vers ce paquetage

Architecture logique

Qu'est-ce qu'une architecture logique ?

- Regroupement des classes logicielles en paquetages
 ~> Point de départ pour un découpage en sous-systèmes

Objectifs d'une architecture logique :

- Encapsuler et décomposer la complexité
- Faciliter le travail en équipes
- Faciliter la réutilisation et l'évolutivité

~> Forte cohésion intra paquetage

~> Faible couplage inter paquetages

Exemples d'architectures logiques :

- Architecture en couches
- Architecture Modèle - Vue - Contrôleur (MVC)
- Architecture Multi-tiers
- ...

Plan du cours

1 Introduction

2 Modéliser la structure avec UML

- Structuration Orientée Objet
- Diagrammes d'objets
- Diagrammes de classes
- Diagrammes de paquetage
- Diagrammes de composants
- Diagrammes de déploiement

3 Modéliser le comportement avec UML

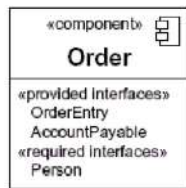
4 Principes et patrons de conception orientée objet

Diagrammes de composants

Composant :

- Encapsule l'état et le comportement d'un ensemble de classiers (classes, composants...)
 - Spécifie les services fournis et requis :
 - Interfaces fournies : «provided interfaces» ou —○
 - Interfaces requises : «required interfaces» ou —○
- ~ Substituable à tout composant qui offre/requiert les m^êm interfaces

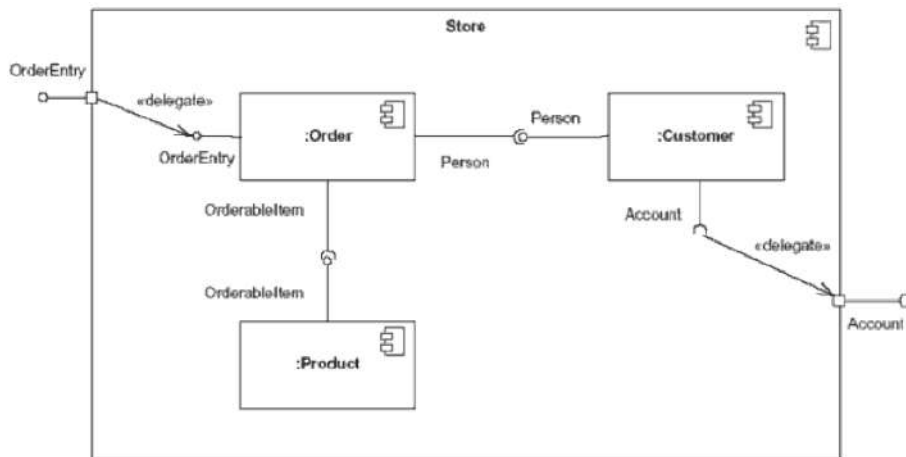
Exemple :



[Image extraite de www.ibm.com/developerworks/rational]

Relations entre composants

~ Inclusion entre composants et Fourniture ou Utilisation d'interfaces



[Image extraite de www.ibm.com/developerworks/rational]

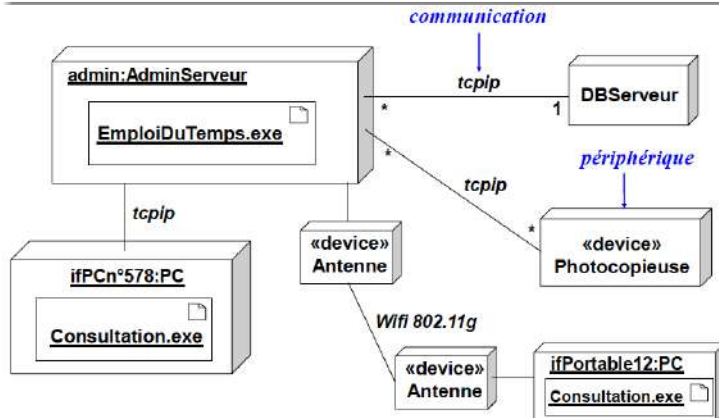
Plan du cours

- 1 Introduction
- 2 **Modéliser la structure avec UML**
 - Structuration Orientée Objet
 - Diagrammes d'objets
 - Diagrammes de classes
 - Diagrammes de paquetage
 - Diagrammes de composants
 - Diagrammes de déploiement
- 3 **Modéliser le comportement avec UML**
- 4 **Principes et patrons de conception orientée objet**

Diagrammes de déploiement

Modéliser le déploiement du système sur une architecture physique

- Disposition des artefacts sur les nœuds physiques
 - Artefacts : instances de composants, processus, ...
 - Nœuds physiques : Ordinateur, Téléphone, Imprimante, ...
- Moyens de communication entre les nœuds



Plan du cours

- 1 Introduction
- 2 Modéliser la structure avec UML
- 3 Modéliser le comportement avec UML**
- 4 Principes et patrons de conception orientée objet

Modéliser le comportement avec UML

Décrire le comportement du système en termes d'interactions

- Système vu comme une boîte noire :
 - Qui interagit avec le système, et dans quel but ?
~> Diagramme de cas d'utilisation
 - Comment interagit le système avec son environnement ?
~> Diagramme de séquence système
- Système vu comme une boîte blanche :
 - Comment interagissent les objets du système ?
~> Diagramme de séquence et/ou de communication

Décrire l'évolution du système dans le temps

- Comment évoluent les états des objets ?
~> Diagrammes d'états-transitions

Plan du cours

- 1 Introduction
- 2 Modéliser la structure avec UML
- 3 **Modéliser le comportement avec UML**
 - Diagrammes d'interaction
 - Diagrammes de cas d'utilisation
 - Diagrammes d'états-transitions
- 4 Principes et patrons de conception orientée objet

Diagrammes d'interaction

~> Point de vue temporel sur les interactions

Pendant la capture des besoins (système = boîte noire) :

~> Interactions entre acteurs et système

- Décrire les scénarios des cas d'utilisation

Pendant la conception (système = boîte blanche) :

~> Interactions entre objets

- Réfléchir à l'affectation de responsabilités aux objets
 - Qui crée les objets ?
 - Qui permet d'accéder à un objet ?
 - Quel objet reçoit un message provenant de l'IHM ?
 - ...

de façon à avoir un faible couplage et une forte cohésion

- Elaboration en parallèle avec les diagrammes de classes

~> Contrôler la cohérence des diagrammes !

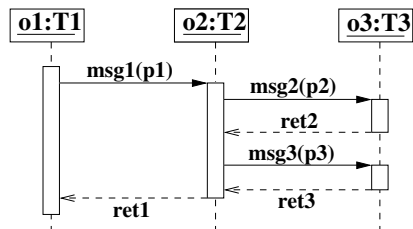
Diagrammes de séquence vs diagrammes de communication

Diagrammes de séquence :

Structuration en termes de

- temps $\leadsto$ axe vertical
- objets $\leadsto$ axe horizontal

Exemple :

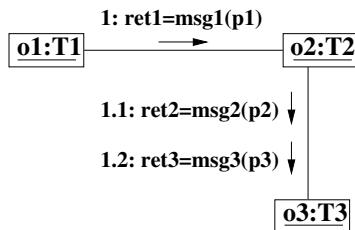


Diagrammes de communication :

Structuration en multigraphe

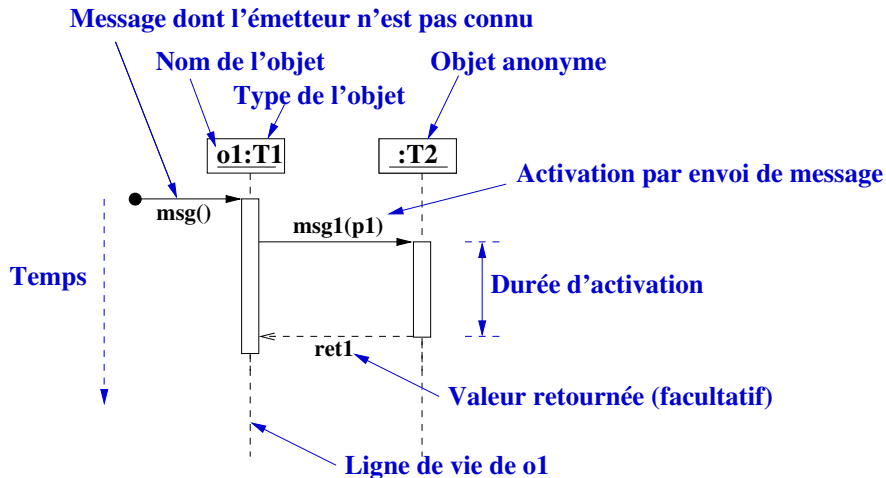
- Numérotation des arcs pour modéliser l'ordre des interactions

Exemple :



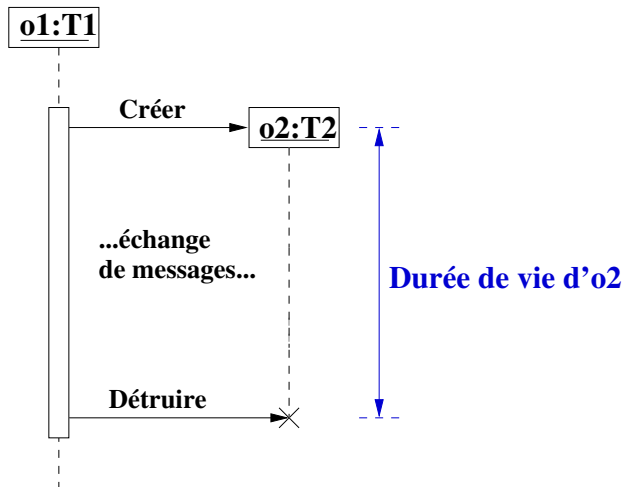
Diagrammes de séquence

Ligne de vie et activation



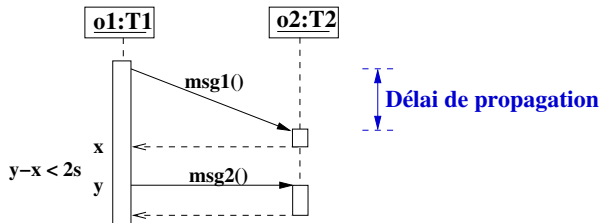
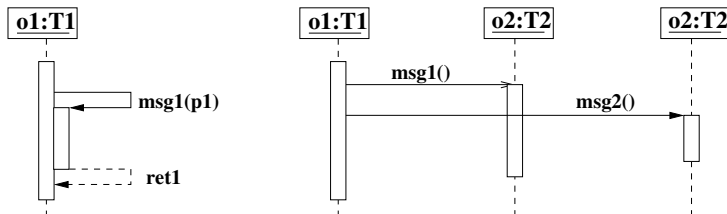
Diagrammes de séquence

Création et destruction d'objets



Diagrammes de séquence

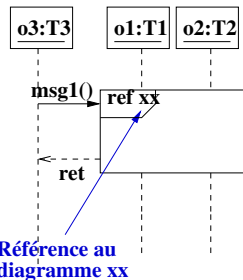
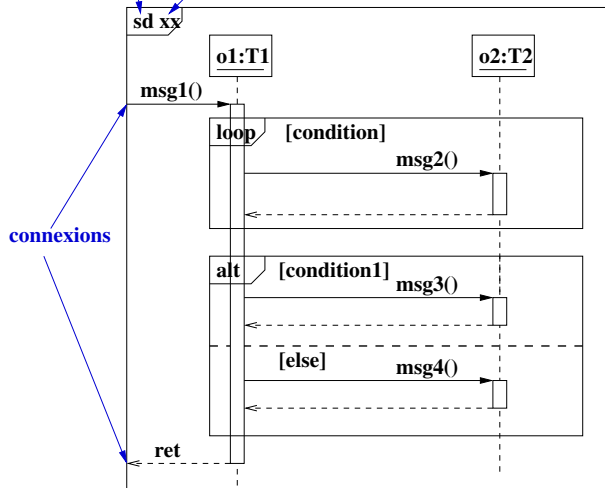
Messages réflexifs, messages asynchrones et contraintes temporelles



Diagrammes de séquence : Cadres

Mot réservé pour déclarer un diagramme de séquence

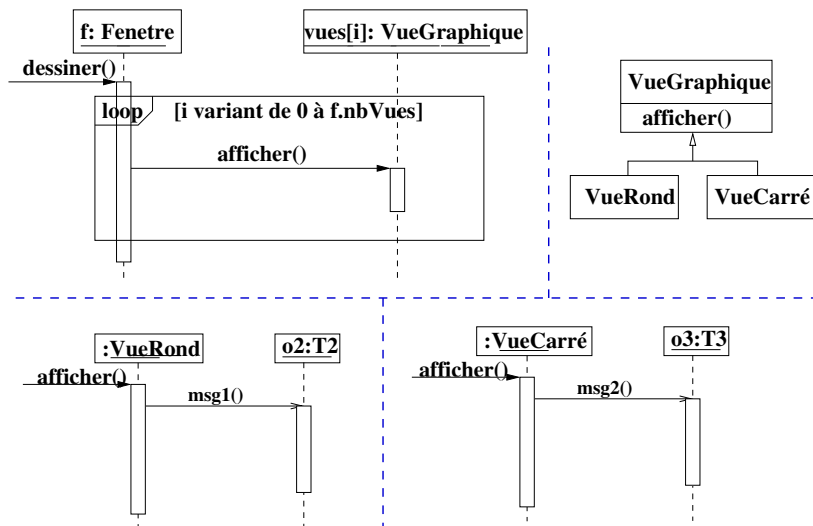
Nom du diagramme de séquence



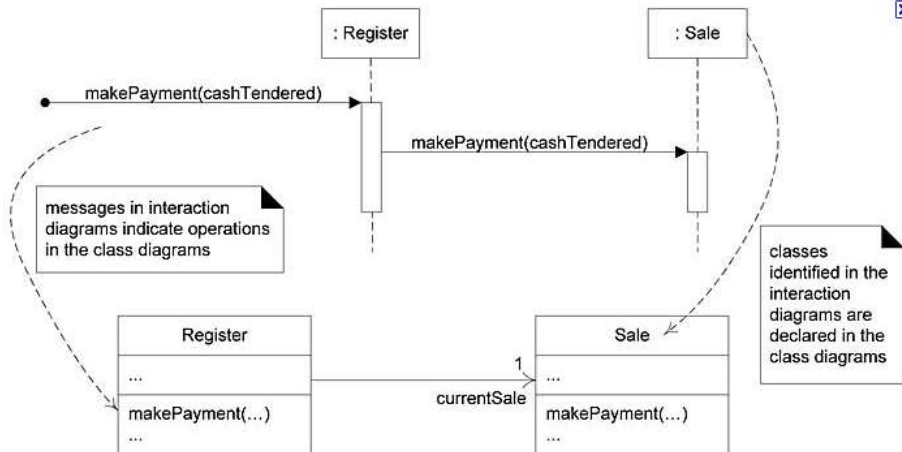
~> Existe aussi : par, opt, critique

Diagrammes de séquence

Messages polymorphes



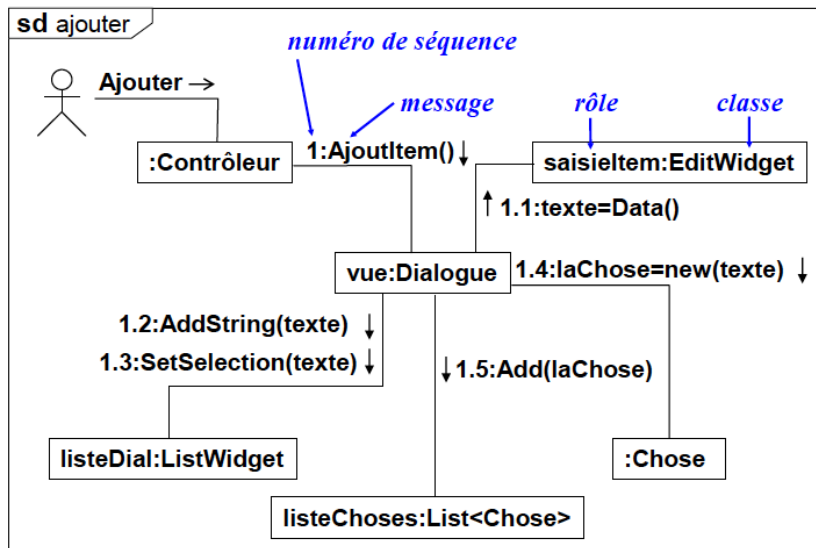
Liens entre diag. de classes et d'interaction



[Figure extraite du livre de C. Larman]

Diagrammes de communication

Présentation alternative d'une séquence d'interactions



Plan du cours

1 Introduction

2 Modéliser la structure avec UML

3 Modéliser le comportement avec UML

- Diagrammes d'interaction
- Diagrammes de cas d'utilisation
- Diagrammes d'états-transitions

4 Principes et patrons de conception orientée objet

Cas d'utilisation

Pourquoi faire ?

Permettre au client de décrire ses besoins

- Parvenir à un accord (contrat) entre clients et développeurs
- Point d'entrée pour les étapes suivantes du développement

Qu'est ce qu'un cas d'utilisation ?

- Usage que des acteurs font du système
 - Acteur : Entité extérieure qui interagit avec le système
 - ↪ Une même personne peut jouer le rôle de différents acteurs
 - ↪ Un acteur peut être un autre système (SGBD, Horloge, ...)
 - Usage : Séquence d'interactions entre le système et les acteurs
- Généralement composé de plusieurs scénarios (instances)
 - ↪ Scénario de base et ses variantes (cas particuliers)
 - ↪ Description des scénarios à l'aide de diagrammes de séquence

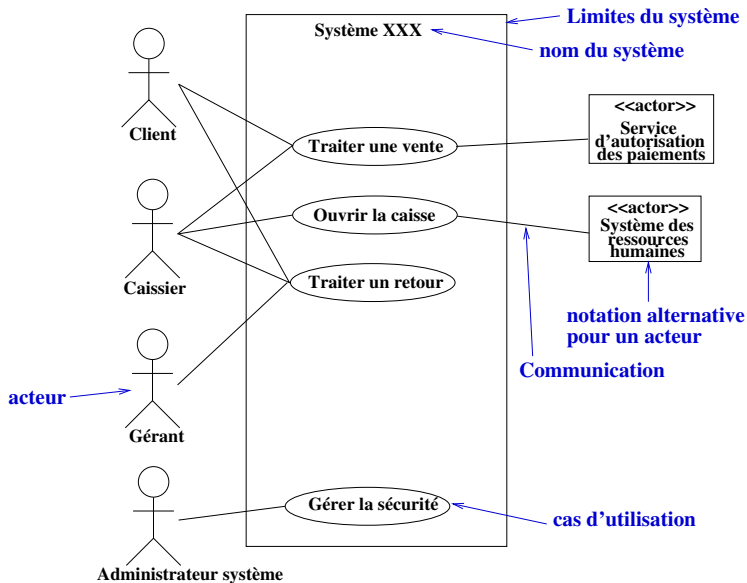
Comment découvrir les cas d'utilisation ?

- Délimiter le périmètre du système
- Identifier les acteurs interagissant avec le système :
 - Ceux qui utilisent le système
 - Ceux qui fournissent un service au système
- Identifier les acteurs principaux
 - Ceux qui utilisent le système **pour atteindre un but**
- Définir les cas d'utilisation correspondant à ces buts
 - Nom = Verbe à l'infinitif + Groupe nominal

Comment décrire les cas d'utilisation ?

- Diagramme de cas d'utilisations
 - Récapitulatif graphique des interactions entre acteurs et cas
- Diagramme de séquence
 - Description de chaque scénario
 - Séquences d'interactions entre les acteurs et le système

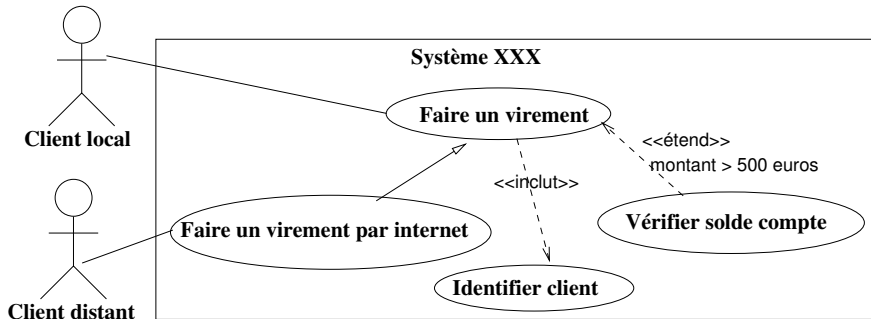
Diagrammes de cas d'utilisation



Relations entre cas d'utilisation

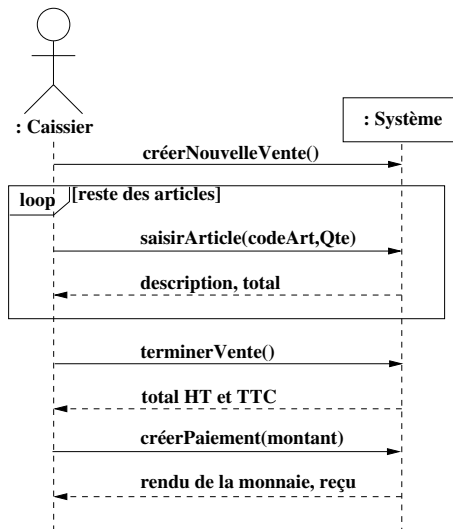
- Généralisation : ————▷
- Inclusion : - - - - «include» - - - - >
- Extension : - - - - «extend» - - - - > (préciser la condition)

~> A utiliser avec modération



Diagrammes de séquence d'un cas d'utilisation

Représentation graphique d'un scénario d'un cas d'utilisation



~> Le système vu comme une boîte noire

Plan du cours

- 1 Introduction
- 2 Modéliser la structure avec UML
- 3 Modéliser le comportement avec UML
 - Diagrammes d'interaction
 - Diagrammes de cas d'utilisation
 - Diagrammes d'états-transitions
- 4 Principes et patrons de conception orientée objet

Parenthèse sur les automates finis (1/3)

Un automate fini est défini par

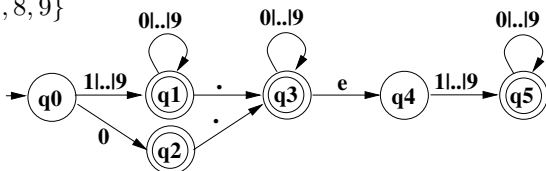
- Un ensemble de symboles $\mathcal{A}$
- Un ensemble fini d'états Q
- Un ensemble d'états initiaux $I \subseteq Q$ et d'états terminaux $T \subseteq Q$
- Une relation de transition $R \subseteq Q \times \mathcal{A} \times Q$
 - Interprétation : $(q_i, s, q_j) \Rightarrow$ passer de q_i à q_j quand on lit s

Représentation d'un automate fini par un graphe

- Chaque état correspond à un sommet
- Chaque transition (q_i, s, q_j) correspond à un arc $q_i \xrightarrow{s} q_j$

Exemple :

- $\mathcal{A} = \{e, ., 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$
- $T = \{q_1, q_2, q_3, q_5\}$
- $I = \{q_0\}$



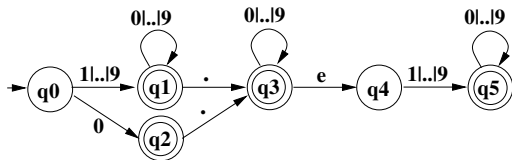
Parenthèse sur les automates finis (2/3)

Exécution d'un automate fini :

- Un automate est une procédure qui reconnaît des mots
 - Entrée = une suite de symboles de $\mathcal{A}$
 - Sortie = vrai ou faux
- Un mot $\langle s_1, \dots, s_n \rangle$ est reconnu s'il existe $\langle q_0, \dots, q_n \rangle$ tel que :
 $q_0 \in I, q_n \in T$ et $\forall i \in [1..n], (q_{i-1}, s_i, q_i) \in R$
 $\leadsto$ Chemin partant d'un état de I et arrivant sur un état de F

Exemple :

- Mots reconnus :
0, 0.123e45, 125, ...
- Mots non reconnus :
012, 4.5.6, 1e2, ...



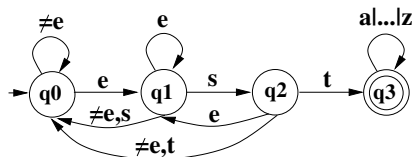
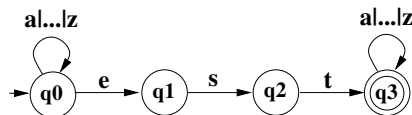
Comment modifier l'automate pour qu'il accepte 1e2 ?

Parenthèse sur les automates finis (3/3)

Automates finis déterministes et complets

- Un automate est déterministe si R est une fonction de $Q \times \mathcal{A}$ dans Q
 $\leadsto$ On peut toujours rendre déterministe un automate
- Un automate est complet si R est une fonction totale
 $\leadsto$ On peut toujours rendre complet un automate

Exemple :



Au delà des automates finis

Les automates finis déterministes sont très efficaces

- Complexité en temps linéaire / taille du mot

Mais ils ont un pouvoir d'expression limité aux langages réguliers

- Ne peuvent reconnaître les expressions bien parenthésées (entre autres)

Un automate à pile a un pouvoir d'expression supérieur

- A chaque transition, on peut empiler ou dépiler des symboles
- Permet de reconnaître tous les langages hors-contexte
 $\leadsto$ Langages de programmation (C++, Java, ...)
- Mais certains langages ne peuvent pas être reconnus par un automate à pile

Une machine de Turing est un automate encore plus puissant

- A chaque transition, on peut écrire ou lire sur un ruban
- Permet de reconnaître tous les langages « décidables »
 $\leadsto$ Pouvoir de calcul équivalent à un ordinateur
- Mais certains langages sont indécidables !

Retour aux diagrammes d'états-transitions

Utilisés pour modéliser :

- Le cycle de vie des objets
 - Evolution de l'état des objets
 - Comportement face à l'arrivée d'événements
- ~> Intéressant si les réponses aux événements dépendent des états
- Mais aussi : protocoles complexes (GUI, ...), processus métier, ...

Pouvoir d'expression variable :

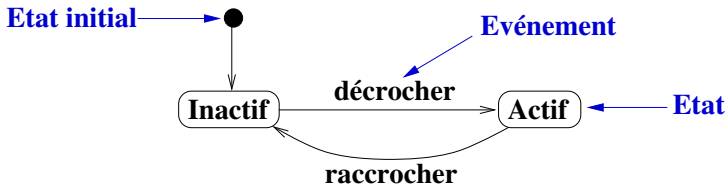
- Pas d'action ni de garde
 - ~> Automates finis
- Pas de garde + actions limitées à l'utilisation d'une pile
 - ~> Automates à pile
- Machines de Turing dans le cas général

Diagrammes sans gardes ni actions

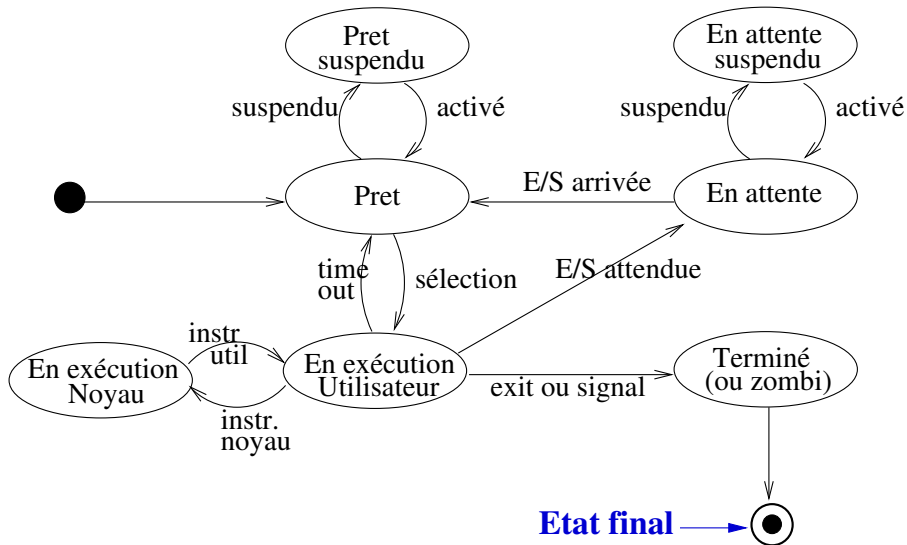
Automates finis un peu particuliers :

- Remplacement de la lecture de symboles par des événements :
 - ↪ Occurrence d'un fait significatif ou remarquable
 - Réception d'un signal
 - Réception d'un message
 - Expiration d'une temporisation
 - ...
- Les événements déclenchent les transitions d'un état vers un autre
 - ↪ Événement «perdu» si aucune transition spécifiée pour lui
- Il y a un état initial, mais pas toujours d'état final

Exemple :

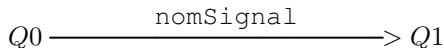


Modélisation des états d'un processus

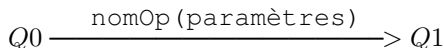


Différents types d'événements :

- Signaux :

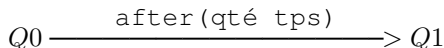


- Appels d'opérations :



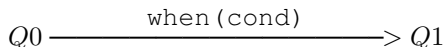
~ Les paramètres peuvent être typés ou non

- Événements temporels :



~ Passage dans l'état $Q1$ `qté tps` après l'arrivée dans l'état $Q0$

- Événements de changement :



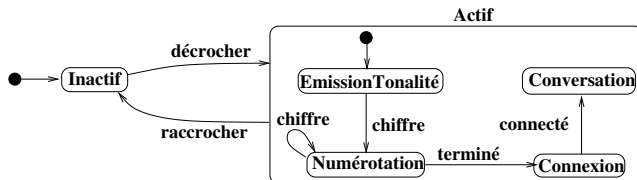
~ Passage dans l'état $Q1$ quand `cond` devient vraie

Etats imbriqués (ou composites)

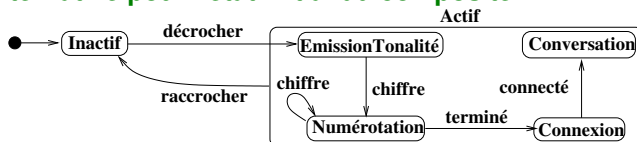
Un état peut contenir des sous-états

- Permet de factoriser les transitions de sortie du composite
 ~> Chaque transition de sortie s'applique à tous les sous-états
- Une seule transition d'entrée

Exemple :



Notation alternative pour l'état initial du composite :

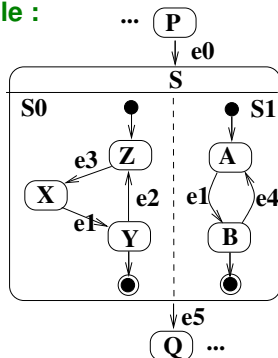


Etats concurrents (1/2)

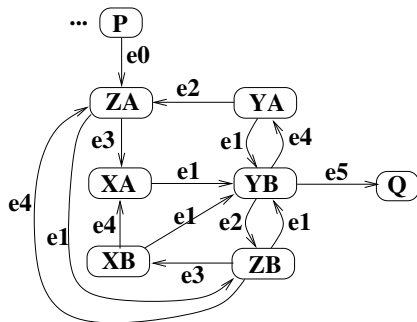
Plusieurs sous-automates peuvent être concurrents :

- Etat courant = n-uplet de sous-états
 $\leadsto$ Etats de $S = \{(Z, A), (Z, B), (X, A), (X, B), (Y, A), (Y, B)\}$
- Exécution indépendante des sous-automates
- Un evt peut déclencher une transition dans plusieurs sous-auto.
- Sortie possible quand tous les sous-auto. sont dans un état final

Exemple :



$\Leftrightarrow$

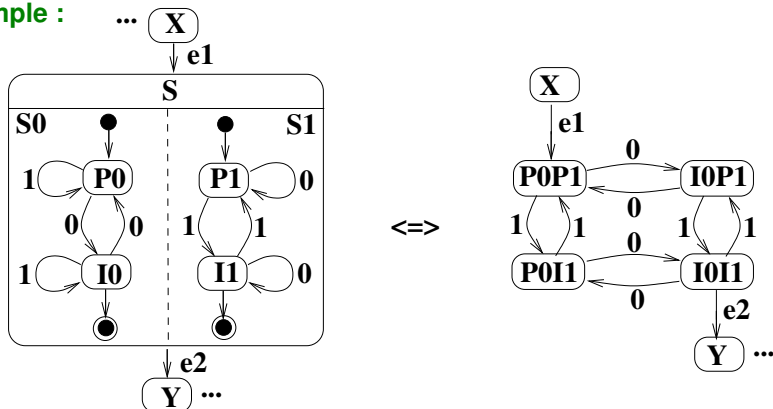


Etats concurrents (2/2)

Cas où les sous-automates sont complets :

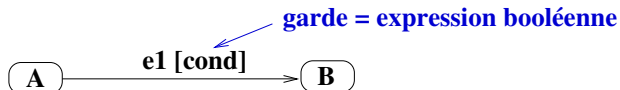
- Equivalent au produit d'automates finis
 $\leadsto$ Intersection des langages reconnus par les sous-automates

Exemple :



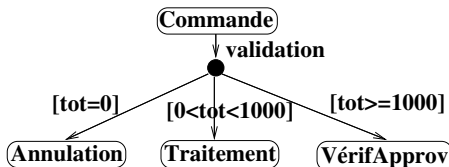
Gardes et transitions composites

Conditions de garde :



- Transition de A vers B réalisée si `cond` est vraie quand `e1` arrive
~ Si `cond` est fausse alors `e1` est perdu

Transitions composites :



- Factorisation de l'événement déclencheur `validation`
- Les gardes doivent être mutuellement exclusives pour que l'automate soit déterministe

Actions et activités

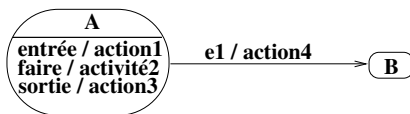
Actions (envoi de signaux, invocation d'opérations, ...) :

- Peuvent être exécutées :
 - Lors d'une transition (ex. : `action4`)
 - En entrant dans un état (ex. : `action1`)
 - En sortant d'un état (ex. : `action3`)
- Sont atomiques (ne peuvent être interrompues par un événement)

Activités :

- Peuvent être exécutées dans un état (ex. : `activité2`)
- Peuvent être continues ou non
- Sont interrompues à l'arrivée d'un événement en sortie de l'état

Exemple :



Ordre d'exécution : `action1` - `activité2` - `action3` - `action4`

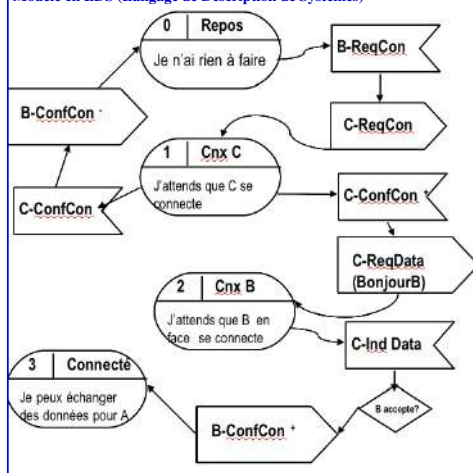
Quelques conseils...

- Pas de transition sans événement
- L'automate doit être déterministe (en général...)
 - Si plusieurs transitions partant d'un état ont le même événement, alors il doit y avoir des gardes qui garantissent le déterminisme
- Tous les états doivent être accessibles depuis l'état initial
- S'il y a des états terminaux alors, pour chaque état non terminal, il doit exister un chemin de cet état vers un état terminal (...en général)

Exercice 1 : traduction d'un modèle en LDS

Connexion entre couches du point de vue de la couche B [F. Biennier]

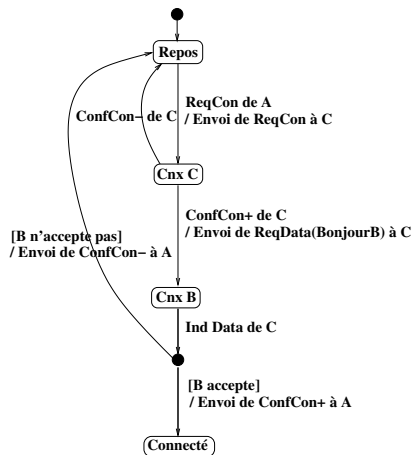
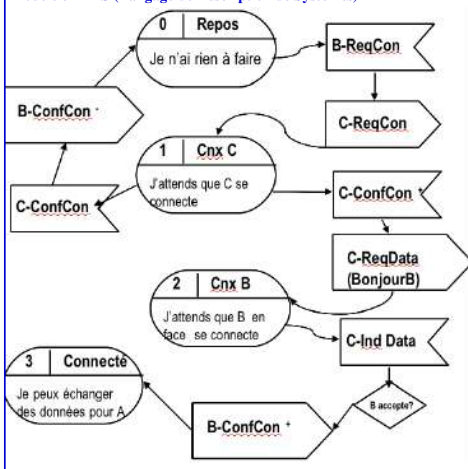
Modèle en LDS (Langage de Description de Systèmes)



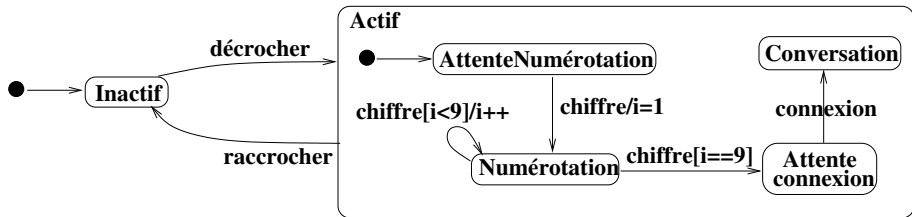
Exercice 1 : traduction d'un modèle en LDS

Connexion entre couches du point de vue de la couche B [F. Biennier]

Modèle en LDS (Langage de Description de Systèmes)



Exercice 2 : Fonctionnement d'un téléphone



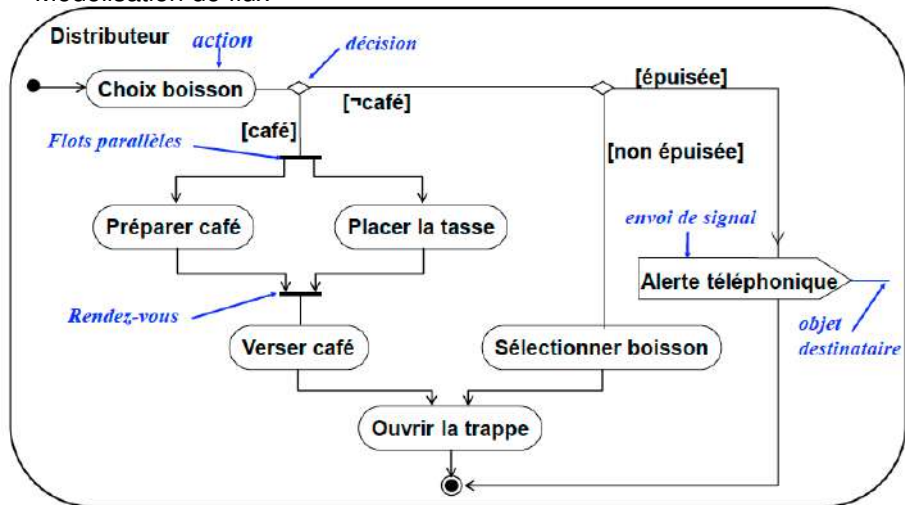
Compléter ce diagramme :

- Emission d'une tonalité quand on décroche
- Emission d'un bip quand on compose un chiffre
- Cas d'un faux numéro
- ...

Diagrammes d'activités

Variante des diagrammes d'états-transitions

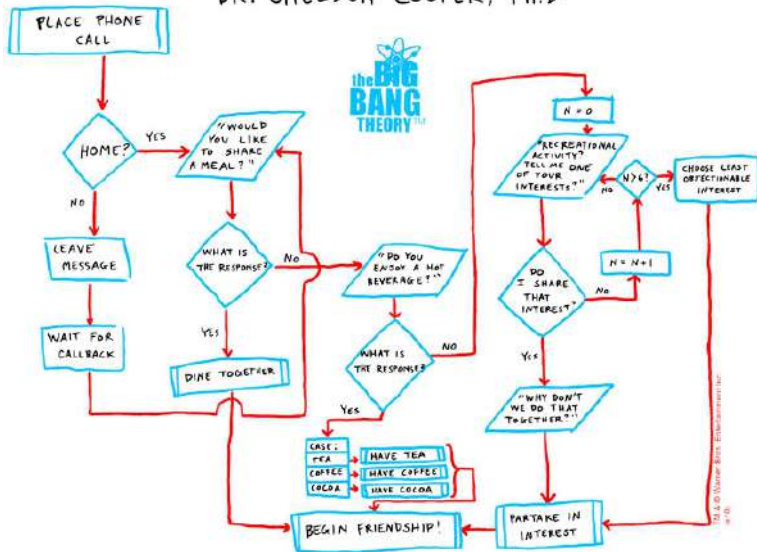
→ Modélisation de flux



[Image empruntée à J.-L. Sourrouille]

THE FRIENDSHIP ALGORITHM

DR. SHELDON COOPER, Ph.D



Plan du cours

- 1 Introduction
- 2 Modéliser la structure avec UML
- 3 Modéliser le comportement avec UML
- 4 Principes et patrons de conception orientée objet**

De UML à la conception orientée objet

Drawing UML diagrams is a reflection of making decisions about the object design. The object design skills are what really matter, rather than knowing how to draw UML diagrams.

Fundamental object design requires knowledge of :

- *Principles of responsibility assignments*
- *Design patterns*

[Extrait du livre de C. Larman]

Principes de conception orientée objet

Ou comment concevoir des logiciels maintenables et réutilisables

Protection des variations : Identifier les points de variation et d'évolution, et séparer ces aspects de ceux qui demeurent constants

Faible couplage : Réduire l'impact des modifications en affectant les responsabilités de façon à minimiser les dépendances entre classes

Forte cohésion : Faciliter la compréhension, gestion et réutilisation des objets en concevant des classes à but unique

Indirection : Limiter le couplage et protéger des variations en ajoutant des objets intermédiaires

Composer au lieu d'hériter : Limiter le couplage en utilisant la composition (boite noire) au lieu de l'héritage (boite blanche) pour déléguer une tâche à un objet

Programmer pour des interfaces : Limiter le couplage et protéger des variations en faisant abstraction de l'implémentation des objets

Ces principes se retrouvent dans beaucoup de Design Patterns...

Patrons de conception (Design patterns)

Patrons architecturaux vs patrons de conception

- Patrons architecturaux : Structuration globale en paquetages
 - ↪ Couches, MVC, Client-serveur, Multi-tiers, ...
- Patrons de conception : Structuration détaillée en classes
 - Attributs et opérations des classes : qui doit savoir, qui doit faire ?
 - Relations entre classes : délégation, héritage, réalisation, ... ?

Description d'un patron de conception

- Nom ↪ Vocabulaire de conception
- Problème : Description du sujet à traiter et de son contexte
- Solution : Description des éléments, de leurs relations/coopérations et de leurs rôles dans la résolution du problème
 - ↪ Description générique
 - ↪ Illustration sur un exemple
- Conséquences : Effets résultant de la mise en œuvre du patron
 - ↪ Complexité en temps/mémoire, impact sur la flexibilité, portabilité, ...

23 patrons du Gang of Four (GoF)

[E. Gamma, R. Helm, R. Johnson, J. Vlissides]

Patrons de création

- Ceux qu'on va voir : *Abstract factory, Factory method, Singleton*
- Et les autres : *Prototype, Builder*

Patrons comportementaux

- Ceux qu'on va voir : *Iterator, Strategy, State, Observer, Command*
- Et les autres : *Visitor, Chain of responsibility, Interpreter, Mediator, Memento, Template method*

Patrons structuraux

- Ceux qu'on va voir : *Decorator, Adapter, Facade, Composite*
- Et les autres : *Bridge, Flyweight, Proxy*

Plan du cours

- 1 Introduction
- 2 Modéliser la structure avec UML
- 3 Modéliser le comportement avec UML
- 4 **Principes et patrons de conception orientée objet**
 - Abstract Factory, Factory method, Singleton
 - Iterator, Strategy, State, Observer, Command
 - Adapter, Facade, Decorator, Composite

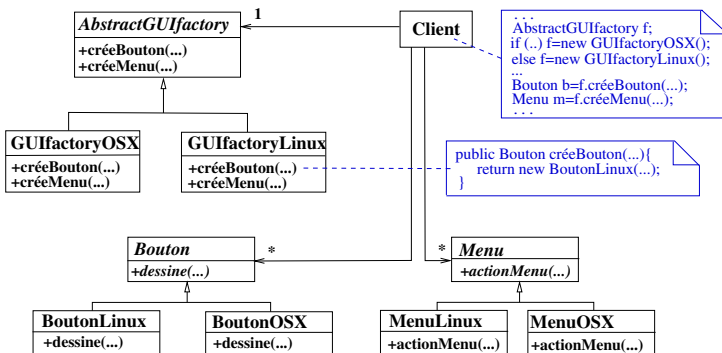
Abstract factory (1/2)

Problème :

Créer une famille d'objets sans spécifier leurs classes concrètes

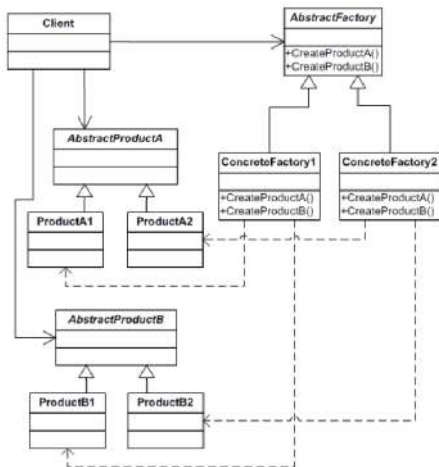
Illustration sur un exemple :

- Créer une interface graphique avec widgets (boutons, menus, ...)
- Point de variation : OS (Linux, OSX, Windows)



Abstract factory (2/2)

Solution Générique [Wikipedia] :



Remarques :

- **AbstractFactory** et **AbstractProduct** sont généralement des interfaces
~ Programmer pour des interfaces
- Les méthodes `createProduct...()` sont des *factory methods*

Avantages du pattern :

- Indirection : Isole Client des implémentations des produits
- Protection des variations : Facilite la substitution de familles de produits
- Maintien automatique de la cohérence

Mais l'ajout de nouveaux types de produits est difficile...

Singleton

Problème :

Assurer qu'une classe possède une seule instance et rendre cette instance accessible globalement

Solution générique [Wikipedia] :

Singleton
- <u>singleton : Singleton</u>
- Singleton()
+ <u>getInstance() : Singleton</u>

```
public static synchronized Singleton getInstance(){  
    if (_singleton == null)  
        _singleton = new Singleton();  
    return _singleton;  
}
```

Exercice :

Utiliser Singleton pour implémenter une classe Factory

Attention :

Parfois considéré comme un anti-pattern... à utiliser avec modération !

Plan du cours

- 1 Introduction
- 2 Modéliser la structure avec UML
- 3 Modéliser le comportement avec UML
- 4 Principes et patrons de conception orientée objet
 - Abstract Factory, Factory method, Singleton
 - Iterator, Strategy, State, Observer, Command
 - Adapter, Facade, Decorator, Composite

Iterator (1/3)

Problème :

Fournir un accès séquentiel aux éléments d'un agrégat d'objets indépendamment de l'implémentation de l'agrégat (liste, tableau, ...)

Illustration sur un exemple en C++ :

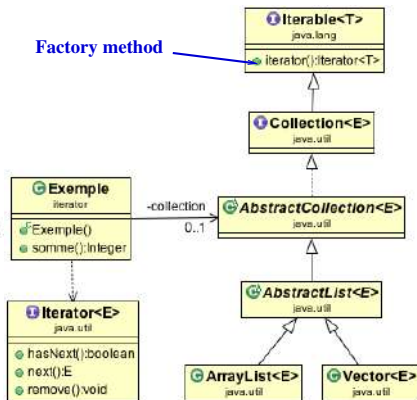
```
class Exemple{
private:
    vector<int> collection;
public:
    int somme(){
        int somme = 0;
        vector<int>::iterator it = collection.begin();
        while (it != collection.end()){
            somme += *it;
            it++;
        };
        return somme;
    }
};
```

Avantage : On peut remplacer `vector<int>` par un autre container sans modifier le code de `somme()`

Iterator (2/3)

Illustration sur un exemple en Java :

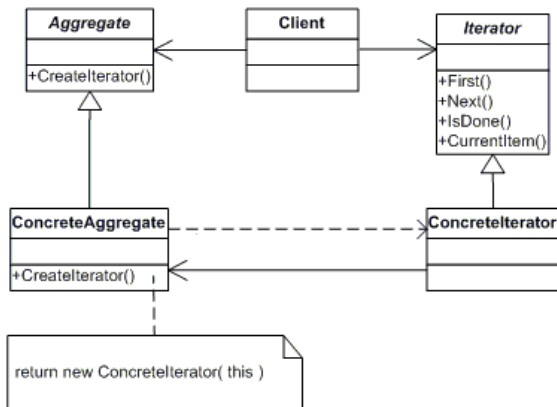
```
public class Exemple {
    private AbstractCollection<Integer> collection;
    public Exemple(){
        collection = new ArrayList<Integer>();
        // ... ajout d'éléments dans collection
    }
    public Integer somme(){
        Integer somme = 0;
        Iterator<Integer> it = collection.iterator();
        while (it.hasNext())
            somme += it.next();
        return somme;
    }
}
```



- Question : Pourquoi séparer `Iterator` de `Collection` ?
- Exercice : Dessiner le diagramme de séquence de `somme()`

Iterator (3/3)

Solution générique :



Avantages :

- Protection des variations : Client est protégé des variations d'Aggregate
- Forte cohésion : Séparation du parcours de l'agrégation
- Possibilité d'avoir plusieurs itérateurs sur un même agrégat en même temps

Strategy (1/3)

Problème :

Changer dynamiquement le comportement d'un objet

Illustration sur un exemple :

- Dans un jeu vidéo, des personnages combattent des monstres...
 ~> méthode `combat (Monstre m)` de la classe `Perso`
...et le code de `combat` peut être différent d'un personnage à l'autre
 - Sol. 1 : `combat` contient un cas pour chaque type de combat
 - Sol. 2 : La classe `Perso` est spécialisée en sous-classes qui redéfinissent `combat`
- Représenter ces solutions en UML. Peut-on facilement :
 - Ajouter un nouveau type de combat ?
 - Changer le type de combat d'un personnage ?

Strategy (1/3)

Problème :

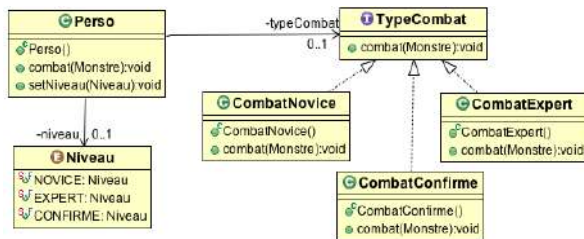
Changer dynamiquement le comportement d'un objet

Illustration sur un exemple :

- Dans un jeu vidéo, des personnages combattent des monstres...
 ~> méthode `combat (Monstre m)` de la classe `Perso`
...et le code de `combat` peut être différent d'un personnage à l'autre
 - Sol. 1 : `combat` contient un cas pour chaque type de combat
 - Sol. 2 : La classe `Perso` est spécialisée en sous-classes qui redéfinissent `combat`
 - Sol. 3 : La classe `Perso` délègue le combat à des classes encapsulant des codes de combat et réalisant toutes une même interface
- Représenter ces solutions en UML. Peut-on facilement :
 - Ajouter un nouveau type de combat ?
 - Changer le type de combat d'un personnage ?

Strategy (2/3)

Diagramme de classes de la solution 3 :



Code Java de la classe Perso :

```

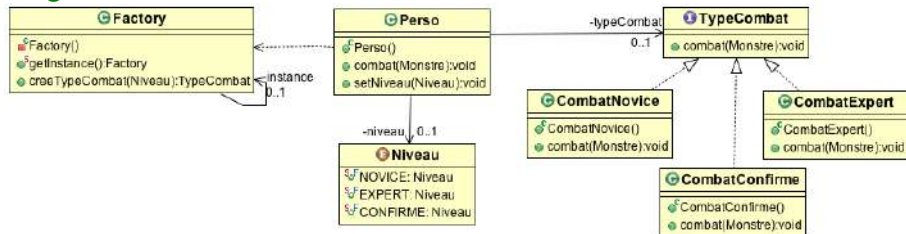
public class Perso {
    private TypeCombat typeCombat;
    private Niveau niveau;
    public Perso(){
        niveau = Niveau.NOVICE;
        typeCombat = 
    }
    public void combat(Monstre m){
        typeCombat.combat(m);
    }
    public void setNiveau(Niveau niveau) {
        this.niveau = niveau;
        typeCombat = 
    }
    // ...Autres méthodes de Perso...
}

```

Comment créer l'instance
de TypeCombat correspondant
à niveau ?

Strategy (2/3)

Diagramme de classes de la solution 3 :



Code Java de la classe Perso :

```

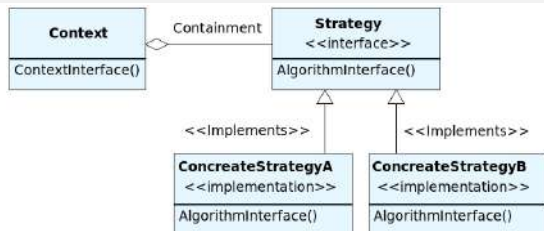
public class Perso {
    private TypeCombat typeCombat;
    private Niveau niveau;
    public Perso(){
        niveau = Niveau.NOVICE;
        typeCombat = Factory.getInstance().creerTypeCombat(niveau);
    }
    public void combat(Monstre m){
        typeCombat.combat(m);
    }
    public void setNiveau(Niveau niveau) {
        this.niveau = niveau;
        typeCombat = Factory.getInstance().creerTypeCombat(niveau);
    }
}
// ...Autres méthodes de Perso...

```

Strategy (3/3)

Solution générique :

[Wikipedia]



Remarques :

- Principes de conception orientée objet mobilisés :
 - Indirection : Isole `Context` des implémentations de `Strategy`
~> Protection des variations
 - Composer au lieu d'hériter : Changer dynamiquement de stratégie
- Passage d'informations de `Context` à `Strategy`
 - en "poussant" : l'info est un param de `AlgorithmInterface()`
 - en "tirant" : le contexte est un param. de `AlgorithmInterface()` qui utilise des getters pour récupérer l'info

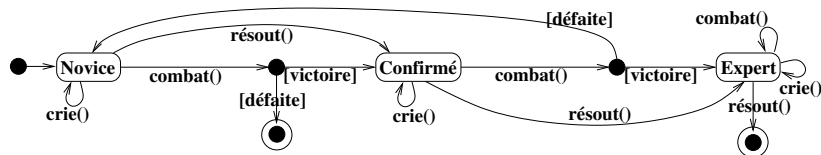
State (1/3)

Problème :

Modifier le comportement d'un objet en fonction de son état

Illustration sur un exemple :

Les personnages d'un jeu peuvent combattre, résoudre des énigmes et crier :



- Sol. 1 : Chaque méthode de `Perso` contient un cas par état

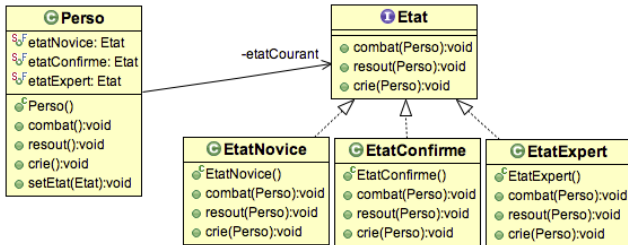
```
public void resout() {  
    if (etat == NOVICE) { ...; etat = CONFIRME; }  
    else if (etat == CONFIRME) { ...; etat = EXPERT; }  
    else { ...; System.exit(0); }  
}
```

- Quel est le coût de l'ajout d'un nouvel état ou d'une nouvelle action ?

State (2/3)

Solution 2 :

Encapsuler les états dans des classes implémentant une même interface



```

public class Perso {
    public static final Etat etatNovice = new EtatNovice();
    public static final Etat etatConfirme = new EtatConfirme();
    public static final Etat etatExpert = new EtatExpert();
    private Etat etatCourant;
    public Perso(){ etatCourant = etatNovice; }
    public void combat(){ etatCourant.combat(this); }
    public void resout(){ etatCourant.resout(this); }
    public void crie(){ etatCourant.crie(this); }
    public void setEtat(Etat e) { etatCourant = e; }
}
  
```

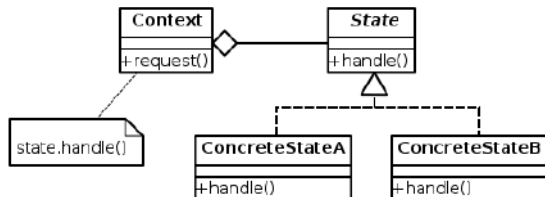
```

public class EtatConfirme implements Etat {
    public void combat(Perso p) {
        // ...code de combat de confirmé
        if (gagnant)
            p.setEtat(Perso.etatExpert);
        else
            p.setEtat(Perso.etatNovice);
    }
    // ...
}
  
```

State (3/3)

Solution générique :

[Wikipedia]



Remarques :

- Ajout d'un nouvel état facile...
...mais ajout d'une nouvelle action plus compliqué
- Si `ConcreteState` ne mémorise pas d'information interne
Alors les états peuvent être des attributs statiques de `Context`
Sinon il faut une instance de `ConcreteState` par instance de `Context`
~> Peut devenir coûteux en mémoire !
- Point commun avec Strategy : utilise la délégation pour modifier dynamiquement le comportement des instances de `Context`, comme si elles changeaient de classes

Observer (aka Publish/Subscribe) (1/3)

Problème :

Faire savoir à un ensemble d'objets (observateurs/abonnés) qu'un autre objet (observable/publieur) a été modifié

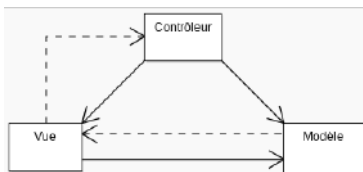
Illustration sur l'exemple du jeu vidéo :

La représentation (vue) des personnages dépend de leur niveau...

...et on peut avoir plusieurs vues (graphique, sonore, textuelle, etc)

...et on veut se protéger des évolutions et variations sur ces vues

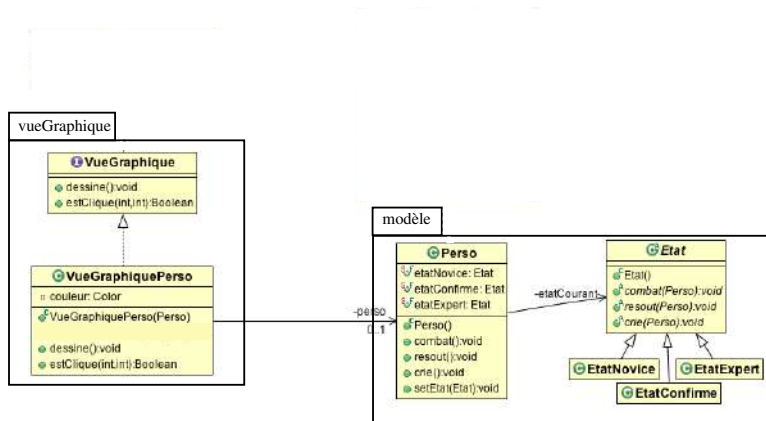
~> Utilisation du patron architectural Model-View-Controller (MVC)



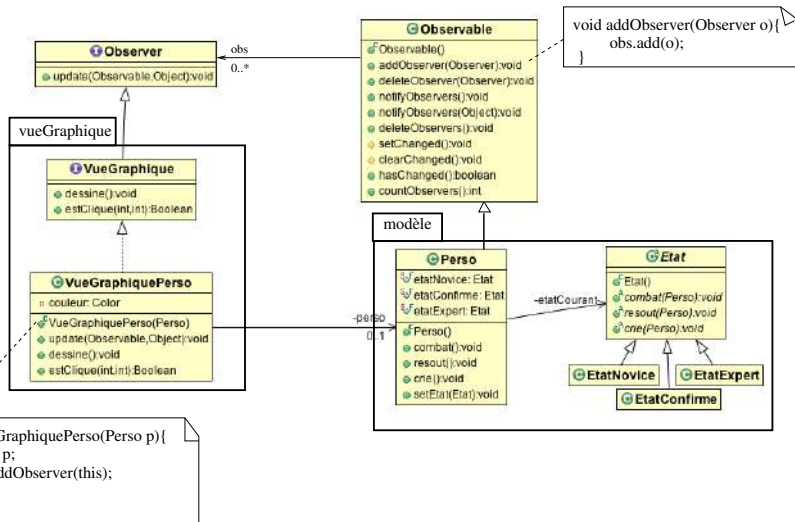
- Modèle : traite les données
- Vue : affiche les données, reçoit les evt clavier/souris et les envoie au contrôleur
- Contrôleur : analyse les evt clavier/souris et active le modèle et/ou la vue en conséquence

Modèle est observable, Vue est observateur

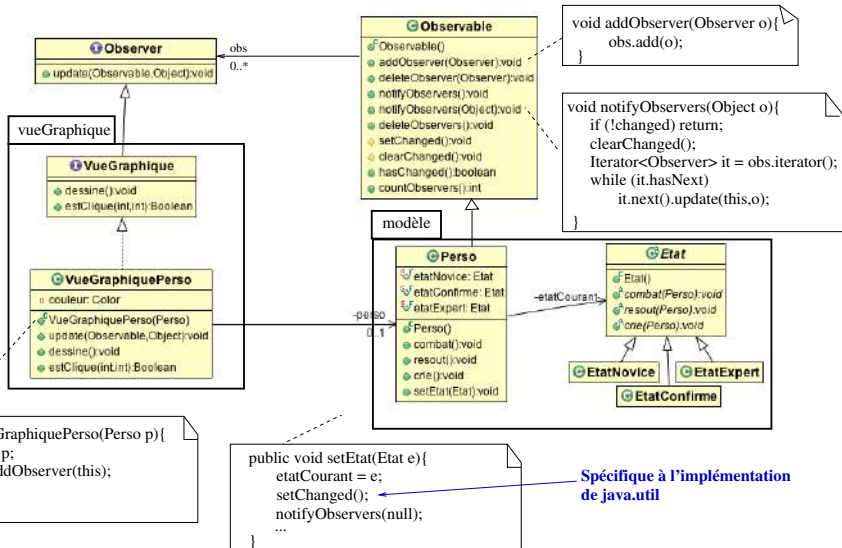
Observer (aka Publish/Subscribe) (2/3)



Observer (aka Publish/Subscribe) (2/3)

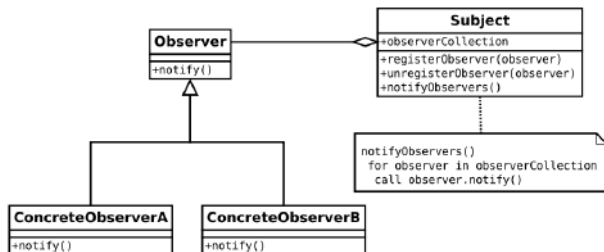


Observer (aka Publish/Subscribe) (2/3)



Observer (aka Publish/Subscribe) (3/3)

Solution générique [Wikipedia] :



Remarques :

- Faible couplage entre `ConcreteObserver` et `Subject`
- Les données de `Subject` peuvent être “poussées” (dans `notify`) ou “tirées” (avec des `getters`)
- Se retrouve dans de nombreuses API Java
 ↪ “Listeners” de l’API Swing pour observer le clavier, la souris, ...

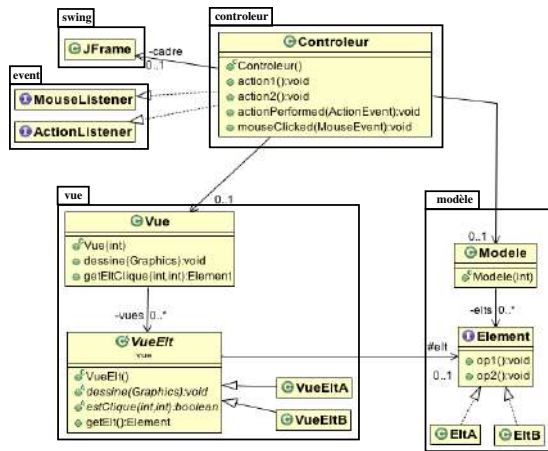
Command (1/3)

Problème :

Découpler la réception d'une requête de son exécution

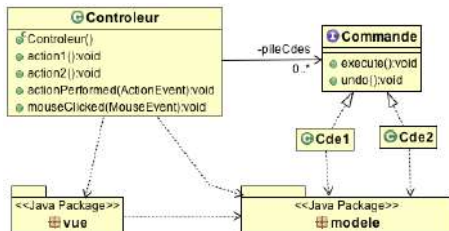
Illustration / modèle MVC :

- Le contrôleur reçoit les événements utilisateur (`actionPerformed` et `mouseClicked`) et appelle en conséquence des méthodes (`action1` et `action2`) qui activent le modèle et la vue
- On veut garder un historique pour pouvoir annuler les dernières actions



Command (2/3)

- Encapsuler les commandes dans des objets contenant les informations permettant de les exécuter/annuler
- Stocker les commandes dans une pile



```

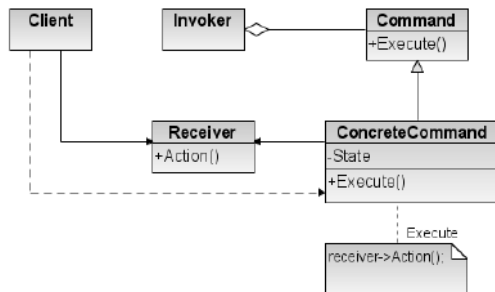
public class Controleur
implements ActionListener, MouseListener{
    private Modele modele;
    private Vue vue;
    private Stack<Commande> pileCdes;
    public Controleur() {
        modele = new Modele(paramsModele);
        vue = new Vue(paramsVue);
        pileCdes = new Stack<Commande>();
    }
    public void action1() {
        // ...
        Commande cde = new Cde1(paramsCde);
        pileCdes.push(cde);
        cde.execute();
    }
    public void undo() {
        if (!pileCdes.empty()){
            Commande c = pileCdes.pop();
            c.undo();
        }
        else System.out.println("Pas de cde à annuler");
    }
    // ...
}

```

Command (3/3)

Solution générique :

- `Client` crée les instances de `ConcreteCommand`
- `Invoker` demande l'exécution des commandes
- `ConcreteCommand` délègue l'exécution à `Receiver`



Remarques :

- Découple la réception d'une requête de son exécution
- Les rôles de `Client` et `Invoker` peuvent être joués par une même classe (par exemple le contrôleur)
- Permet la journalisation des requêtes pour reprise sur incident
- Permet d'annuler ou ré-exécuter des requêtes (undo/redo)

Plan du cours

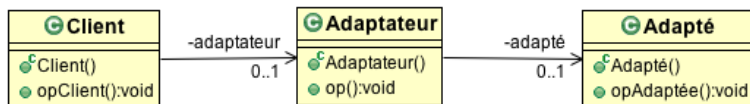
- 1 Introduction
- 2 Modéliser la structure avec UML
- 3 Modéliser le comportement avec UML
- 4 Principes et patrons de conception orientée objet
 - Abstract Factory, Factory method, Singleton
 - Iterator, Strategy, State, Observer, Command
 - Adapter, Facade, Decorator, Composite

Adapter

Problème :

Fournir une interface stable (Adaptateur) à un composant dont l'interface peut varier (Adapté)

Solution générique :



→ Application des principes “indirection” et “protection des variations”

Exercices :

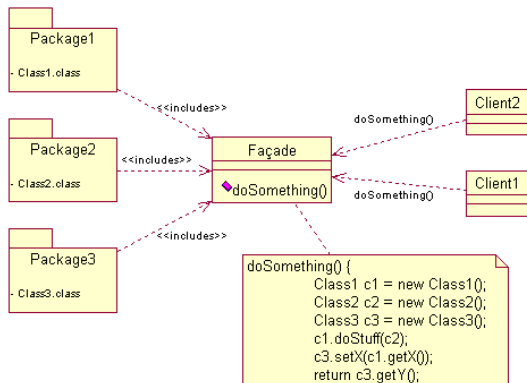
- Dessiner le diagramme de séquence de l'envoi du message `opClient()` à une instance de `Client`
- Comment faire s'il y a plusieurs composants (`Adapté`) différents, et que l'on veut pouvoir choisir dynamiquement la classe adaptée ?

Facade

Problème :

Fournir une interface simplifiée (Facade)

Solution générique [Wikipedia] :



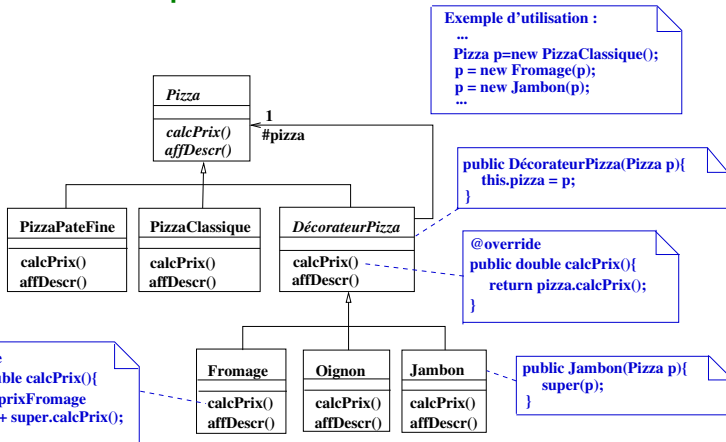
→ Application des principes “indirection” et “protection des variations”

Decorator (1/2)

Problème :

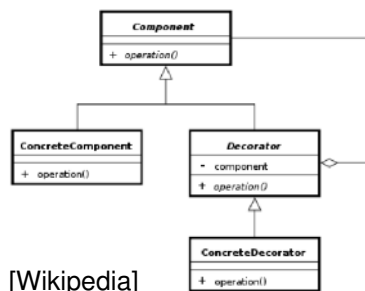
Attacher dynamiquement des responsabilités supplémentaires à un objet

Illustration sur un exemple :



Decorator (2/2)

Solution générique :



[Wikipedia]

Remarques :

- Composer au lieu d'hériter : Ajout dynamique de responsabilités à **ConcreteComponent** sans le modifier
- n décors $\Rightarrow 2^n$ combinaisons
- **Inconvénient** : Peut générer de nombreux petits objets "enveloppes"

Utilisation pour décorer les classes d'entrée/sortie en Java :

- **Component** : `InputStream`, `OutputStream`
- **ConcreteComponent** : `FileInputStream`, `ByteArrayInputStream`, ...
- **Decorator** : `FilterInputStream`, `FilterOutputStream`
- **ConcreteDecorator** : `BufferedInputStream`, `CheckedInputStream`, ...

Adapter, Facade et Decorator

Points communs :

- Indirection $\leadsto$ Enveloppe (wrapper)
- Protection des variations

Différences :

- Adapter : Convertit une interface en une autre (attendue par un Client)
- Facade : Fournit une interface simplifiée
- Decorator : Ajoute dynamiquement des responsabilités aux méthodes d'une interface sans la modifier

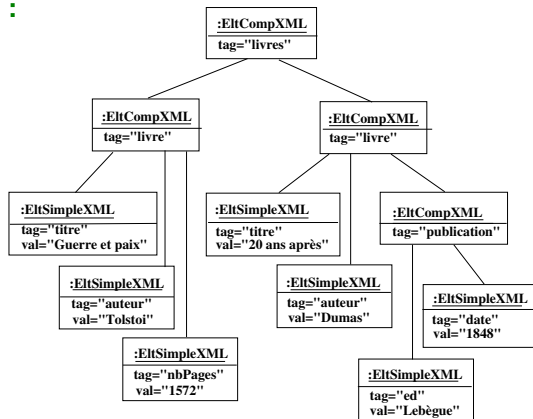
Composite (1/2)

Problème :

Représenter des hiérarchies composant/composé et traiter de façon uniforme les composants et les composés

Illustration sur un exemple :

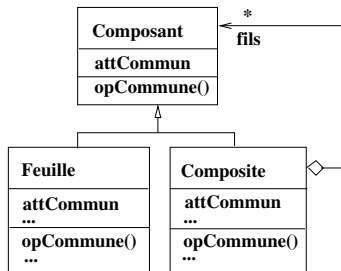
```
< ?xml version="1.0" ?>
< livres>
  < livre>
    < titre>Guerre et paix</titre>
    < auteur>Tolstoï</auteur>
    < nbPages>1572</nbPages>
  </livre>
  < livre>
    < titre>20 ans après</titre>
    < auteur>Dumas</auteur>
    < publication>
      < ed>Lebègue</ed>
      < date>1848</date>
    </publication>
  </livre>
</livres>
```



Comment compter le nombre de tags ?

Composite (2/2)

Solution générique :



Exercices :

- Définir les opérations permettant de :
 - Compter le nombre de fils d'un composant
 - Compter le nombre de descendants d'un composant
 - Ajouter un fils à un composant
- Comment accéder séquentiellement aux fils d'un Composite ?
- Comment accéder séquentiellement aux descendants d'un Composite ?

Autres patterns ?

Patterns de création

- Prototype
- Builder

Patterns structuraux

- Bridge
- Flyweight
- Proxy

Patterns comportementaux

- Visitor
- Chain of responsibility
- Interpreter
- Mediator
- Memento
- Template method